

Article

Real-Time Detection of Rear Car Signals for Advanced Driver Assistance Systems Using Meta-Learning and Geometric Post-Processing

Vasu Tammiseti ^{1,2} , Georg Stettinger ¹ , Manuel Pegalajar Cuellar ²  and Miguel Molina-Solana ^{2,*} ¹ Infineon Technologies AG, 85579 Munich, Germany; vasu.tammiseti@infineon.com (V.T.); georg.stettinger@infineon.com (G.S.)² Department of Computer Science and Artificial Intelligence, University of Granada, 18071 Granada, Spain; manupc@decsai.ugr.es

* Correspondence: miguelmolina@ugr.es

Abstract

Accurate identification of rear light signals in preceding vehicles is pivotal for Advanced Driver Assistance Systems (ADAS), enabling early detection of driver intentions and thereby improving road safety. In this work, we present a novel approach that leverages a meta-learning-enhanced YOLOv8 model to detect left and right turn indicators, as well as brake signals. Traditional radar and LiDAR provide robust geometry, range, and motion cues that can indirectly suggest driver intent (e.g., deceleration or lane drift). However, they do not directly interpret color-coded rear signals, which limits early intent recognition from the taillights. We therefore focus on a camera-based approach that complements ranging sensors by decoding color and spatial patterns in rear lights. This approach to detecting vehicle signals poses additional challenges due to factors such as high reflectivity and the subtle visual differences between directional indicators. We address these by training a YOLOv8 model with a meta-learning strategy, thus enhancing its capability to learn from minimal data and rapidly adapt to new scenarios. Furthermore, we developed a post-processing layer that classifies signals by the geometric properties of detected objects, employing mathematical principles such as distance, area calculation, and Intersection over Union (IoU) metrics. Our approach increases adaptability and performance compared to traditional deep learning techniques, supporting the conclusion that integrating meta-learning into real-time object detection frameworks provides a scalable and robust solution for intelligent vehicle perception, significantly enhancing situational awareness and road safety through reliable prediction of vehicular behavior.

Keywords: meta-learning; YOLO (You Only Look Once); Advanced Driver Assistance Systems (ADAS)



Academic Editor: Christos Bouras

Received: 29 July 2025

Revised: 25 October 2025

Accepted: 28 October 2025

Published: 11 November 2025

Citation: Tammiseti, V.; Stettinger, G.; Pegalajar Cuellar, M.; Molina-Solana, M. Real-Time Detection of Rear Car Signals for Advanced Driver Assistance Systems Using Meta-Learning and Geometric Post-Processing. *Appl. Sci.* **2025**, *15*, 11964. <https://doi.org/10.3390/app152211964>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Advanced Driver Assistance Systems (ADAS) represent a significant advancement in vehicle safety, relying on sensor fusion to enable real-time environmental perception and collision avoidance [1]. Rear-end collisions frequently occur due to delayed recognition of brake signals or misunderstood driver intentions. Additionally, a significant number of intersection accidents happen when turn indicators go unnoticed during lane changes or merges. These issues highlight the need for effective systems to accurately detect and

interpret rear signals, which can play a crucial role in reducing the occurrence of such crashes and improving overall road safety.

Discriminating rear-mounted vehicular intent signals—such as brake lights, turn indicators (Left or Right Indicator Signals, LIS or RIS), and hazard warnings—is essential for optimizing ADAS decision-making algorithms, reducing collision risks, and advancing autonomous driving capabilities. As passenger cars dominate road traffic, their rear signals serve as the most frequent and visual indicators of driver intent, making them an ideal area for algorithmic development.

Despite the increasing adoption of advanced depth-estimation and adverse-condition-resilient technologies, monocular cameras remain indispensable for rear-signal classification due to their unique operational and economic advantages [2]. Their cost-efficiency facilitates widespread integration across vehicle tiers, while their ability to concurrently support lane tracking, traffic sign recognition, and rear-signal analysis consolidates multiple ADAS functions into a unified sensor framework. Critically, cameras capture high-resolution chromatic and spatial data, enabling precise decoding of rear-signal attributes such as color gradients (e.g., red brake lights and amber or red turn signals, depending on region), illumination intensity, and temporal activation patterns [3,4]. This capability is vital in high-density traffic scenarios, where rapid detection of rear-signal transitions (e.g., sudden brake activation) directly impacts collision avoidance. Studies indicate that reducing driver reaction times by 0.5–1 s through timely signal recognition can lower rear-end collision risk by 35%, underscoring the practical significance of vision-based systems [5].

Recent computational approaches leverage deep neural architectures; however, they require extensive retraining on large datasets [6,7]. This dependency hampers scalability across heterogeneous driving environments and diverse automotive designs. Meta-learning addresses this limitation by enabling models to adapt to new tasks with minimal data, transferring knowledge from related tasks to accelerate convergence. Modern architectures that integrate Convolutional Neural Networks (CNNs) with attention mechanisms and meta-learning demonstrate improved resilience against glare, motion blur, and partial occlusions. By efficiently generalizing from limited datasets, these frameworks accommodate variability in automotive taillight designs, signal placements, and regulatory standards, enhancing their robustness and adaptability across diverse driving conditions [8,9].

Building upon Meta-YOLOv8 [10], a meta-learning-enhanced framework optimized for signal detection, our proposal employs a meta-trained backbone network with a task-specific post-processing layer (Meta-YOLOv8+PPL framework) to achieve high-precision discrimination of left-indicating signal, right-indicating signal (LIS and RIS, respectively), and brake lights under challenging conditions (e.g., inclement weather, dynamic lighting) [11,12]. The post-processing layer is key to disambiguating between LIS and RIS, which often exhibit near-identical visual features (e.g., color, shape) and spatial adjacency in taillight clusters. By integrating spatial-temporal context—such as the relative positioning of signals within the car's rear geometry—the layer minimizes false positives and misclassifications.

Radar and LiDAR can infer driver intent indirectly by tracking vehicle positions and movements, but they cannot directly interpret color-based signals like brake lights or turn indicators. In contrast, camera-based meta-learning models effectively decode these visual cues, enabling earlier and more accurate driver intent prediction. Integrating both sensor types offers a comprehensive solution for robust and timely rear-signal recognition in ADAS. Furthermore, while our prior work focused on traffic light color recognition, the present study addresses a distinct and more challenging task: rear taillight signal detection. Unlike traffic lights, taillights are small, highly reflective, and bilaterally symmetric, making left/right side assignment non-trivial without geometric reasoning. Addressing these

challenges requires not only new datasets tailored to rear-signal detection but also the design of a post-processing layer (PPL) that explicitly resolves the orientation of turn signals. Together, these contributions highlight the novelty of the present study and its complementary role within the broader ADAS perception landscape.

The rest of this work is organized as follows: Section 2 describes the broader context of object recognition algorithms; Section 3 offers full details of our proposal. Section 4 describes the experimental setup and Section 5 presents the results and discusses its implications.

2. Related Work

To situate our work within the computational paradigm of automotive perception, we first analyze the methodological evolution of object detection architectures that underpin modern vision-based ADAS [13,14]. The domain of vehicular signal recognition has progressed from rudimentary heuristic frameworks to data-driven deep learning systems, yet critical gaps persist in environmental generalization and contextual adaptability. Early approaches relied on handcrafted feature extraction techniques such as Haar cascades and edge detection filters, coupled with static rule-based classifiers. These methods proved brittle under real-world variability in lighting, occlusion, and heterogeneous signal geometries. Contemporary advances leverage deep neural architectures to address these limitations. Convolutional Neural Networks (CNNs) [15] introduced hierarchical feature learning, enabling robust representation of spatial patterns in taillight clusters and static signal attributes. Region-based methodologies like R-CNN and Faster R-CNN advanced localization accuracy through proposal-driven frameworks, while single-shot detectors such as SSD and YOLO variants optimized speed-accuracy trade-offs via unified detection pipelines [16,17]. These architectures have become cornerstones for real-time signal interpretation, particularly in latency-sensitive ADAS applications.

Convolutional Neural Networks (CNNs) [15] are fundamental to the field of computer vision. They have demonstrated strong performance in tasks such as image classification and object detection. Traditional CNNs, such as AlexNet and VGG, use a series of convolutional layers to extract features from images, followed by fully connected layers for classification. While CNNs have been successfully applied to automotive signal detection, they typically require large amounts of labeled data and significant computational power. One of the main drawbacks of conventional (non-meta) CNN models is their poor adaptability to new tasks without retraining on large datasets, which makes them less effective in data-scarce environments or rapidly changing conditions. Moreover, their performance can degrade under real-world challenges such as varying lighting, occlusions, and signal design diversity—conditions commonly encountered in automotive contexts.

Region-based Convolutional Neural Networks (R-CNNs) introduced a major advancement by incorporating region proposals into the detection process [18]. These models segment an image into multiple regions and apply CNNs to each region to detect objects. Subsequent versions, such as Fast R-CNN and Faster R-CNN, streamlined the pipeline by reducing latency and computational burden. Despite these improvements, R-CNN models still have limitations. They often involve complex pipelines and require significant computational resources for region proposal generation and classification. In addition, the performance of R-CNN models can degrade in real-time applications, making them less ideal for on-the-fly detection of vehicle signals in dynamic environments.

Single Shot MultiBox Detectors (SSDs) [19] address the speed limitations of region-based methods by eliminating explicit region proposal steps. SSDs directly predict object classes and bounding boxes in a single pass through the network. This architecture significantly speeds up the detection process while maintaining high accuracy. However, SSD models face challenges in detecting smaller objects and signals due to the fixed grid structure

used for prediction. In addition, SSDs can exhibit high variance in detection performance under different weather and lighting conditions, which are critical factors in vehicle signal detection. The inability to adapt to new tasks or conditions without extensive retraining remains a notable drawback of non-meta-SSD models.

You Only Look Once (YOLO). This family of architectures revolutionized real-time object detection by framing detection as a single regression problem, directly from image pixels to bounding box coordinates and class probabilities. YOLO models such as YOLOv3 and YOLOv4 are renowned for their speed and accuracy, making them a popular choice for small object detection [20]. Despite their strengths, traditional YOLO models have limitations. They can struggle to detect objects in complex, cluttered scenes, and can produce false positives or negatives in challenging scenarios. They also require large amounts of labeled data for training and can be less effective at dealing with variations in signal appearance and environmental conditions.

Despite their advancements, these models remain constrained by persistent challenges. First, supervised training demands extensive labeled datasets encompassing global taillight designs, signal geometries, and edge cases such as degraded signals or adversarial lighting. Second, static architectures struggle to adapt to domain shifts, including regional variations in signal regulations or unconventional signal placements, without resource-intensive retraining. Finally, reliance on single-frame analysis risks misclassification under occlusion or geometric distortions, as spatial context alone may inadequately resolve ambiguities in signal identification. As later explained, our work addresses these limitations through meta-learning-enhanced detection [21], which embeds adaptive inference mechanisms directly into the YOLOv8 architecture. By contextualizing prior model iterations and their constraints, we establish the foundation for our framework's innovations in few-shot adaptation and post-processing refinements, critical for reliable static signal discrimination in dynamic environments [22].

3. Our Proposal

To address persistent challenges in rear vehicle signal detection, we propose an approach based on the Meta-YOLOv8 model, enhanced with a geometric post-processing layer (PPL). Building on meta-learning principles [8,23], our proposal enables robust feature extraction for car rear light signals even with limited training data. Unlike conventional approaches that struggle with catastrophic forgetting and data scarcity [24], Meta-YOLOv8 adapts dynamically to new signal types while retaining prior knowledge. This is critical for real-world use, where new signal designs are constantly introduced by manufacturers, and environmental conditions vary significantly across deployment scenarios. By reducing reliance on large-scale datasets, the model addresses this fundamental constraint of traditional detection systems while maintaining detection accuracy.

Furthermore, through meta-learning optimization [24], Meta-YOLOv8 achieves rapid adaptation to new signal patterns without extensive retraining cycles, a crucial advantage for resource-constrained edge computing deployments requiring real-time inference speeds [25]. This adaptability extends to environmental variations such as lighting changes or weather conditions, where the model progressively refines its detection capabilities through continuous data exposure. Such dynamic learning ensures sustained performance as automotive signaling standards evolve across manufacturers and regions.

Meta-learning is, therefore, especially well-suited for tail-light detection, which involves small, high-intensity objects that exhibit both strong regularities and significant design variation. Across vehicle models, taillights share consistent chromatic bands, bilateral symmetry, and predictable placement within the rear geometry, yet differ in shape, size, and styling. Episodic meta-training (3-way, 3/5/8/10-shot, in our case) exploits these

shared patterns while maintaining balanced class representation, reducing bias toward frequent categories, and encouraging transferable feature learning. In practice, this enables (i) rapid adaptation to unseen tail-light designs with only a few labeled samples, (ii) increased robustness to lighting changes and occlusion in the small-object regime, and (iii) higher recall under class imbalance—outperforming conventional training, as demonstrated by our few-shot results.

3.1. Meta-YOLOv8

The YOLO series has evolved rapidly since YOLOv1 (2016), with YOLOv2–YOLOv4 improving accuracy–speed trade-offs, YOLOv5 gaining wide adoption through its PyTorch implementation, and YOLOv6/YOLOv7 adding industrial-grade optimizations. During our experiments (late 2023–early 2024), YOLOv8 was the latest Ultralytics release, offering two critical advances for taillight detection: (i) an anchor-free head for precise localization of small, high-contrast objects, and (ii) enhanced multi-scale feature fusion for robustness under varied distances and occlusions. These features gave YOLOv8 clear advantages over earlier YOLO versions and other detectors such as SSD and DETR, enabling faster convergence and real-time inference—key for ADAS. Our results confirmed that YOLOv8 provided the best balance of accuracy and latency on edge devices such as Jetson Nano, see Section 5.3. Later releases (YOLOv9, YOLOv10) introduced refinements like GELAN and improved accuracy–speed trade-offs, though they appeared after our experiments and remain less tested in ADAS-specific small-object tasks. Their added complexity may also hinder inference on constrained hardware, making YOLOv8 still the most mature and practical choice for our framework.

Architecturally, Meta-YOLOv8 builds upon YOLOv8’s proven capabilities in real-time object detection while introducing a novel dual-network framework based on Model-Agnostic Meta-Learning (MAML) [8]. By pairing a base model with a clone network through knowledge-sharing mechanisms (see Figure 1), the system is expected to achieve enhanced generalization to unseen signal patterns while minimizing dataset requirements. This approach diverges fundamentally from single-network paradigms, enabling simultaneous retention of existing knowledge and acquisition of new detection competencies.

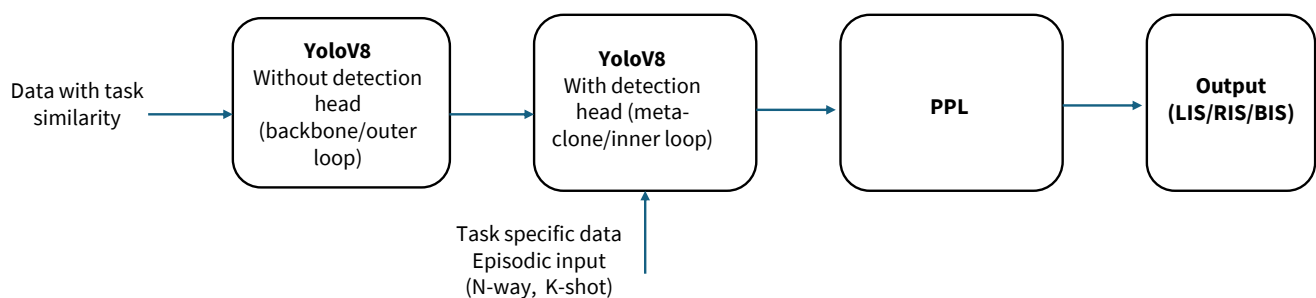


Figure 1. Meta-YOLOv8 Training Schematic. During training, Meta-YOLOv8 creates two temporary model versions: the base model (outer loop) that holds the main weights, and a lighter clone (inner loop) that adapts quickly to small data batches. The clone’s updates refine the base model. After training, the detection head is combined back, and only a single full YOLOv8 model (base model) is used for both testing and inference.

The geometric PPL (Section 3.3) further refines spatial reasoning for signal localization, ensuring precise detection even in cluttered urban traffic scenarios. Combined with YOLOv8's native optimization for inference speed, these innovations position the model as a practical solution for next-generation vehicle safety systems that require both accuracy and computational efficiency.

The Meta-YOLOv8 framework retains YOLOv8's core structural components [26] while introducing targeted enhancements for signal detection tasks. As illustrated in Figure 2, the architecture comprises three principal modules: the backbone for hierarchical feature extraction, the neck for multi-scale feature fusion, and the head for final classification and localization. Key innovations within these modules include optimized C2F (Cross-Stage Partial Bottleneck with 2 Convolutions) blocks for efficient gradient flow, spatial pyramid pooling feature (SPPF) layers for multi-receptive field processing, and strategically placed bottleneck layers to reduce computational overhead [27]. These components, detailed in subsequent sections, collectively enhance the model's capacity to process automotive signal patterns with varying scales and complexities.

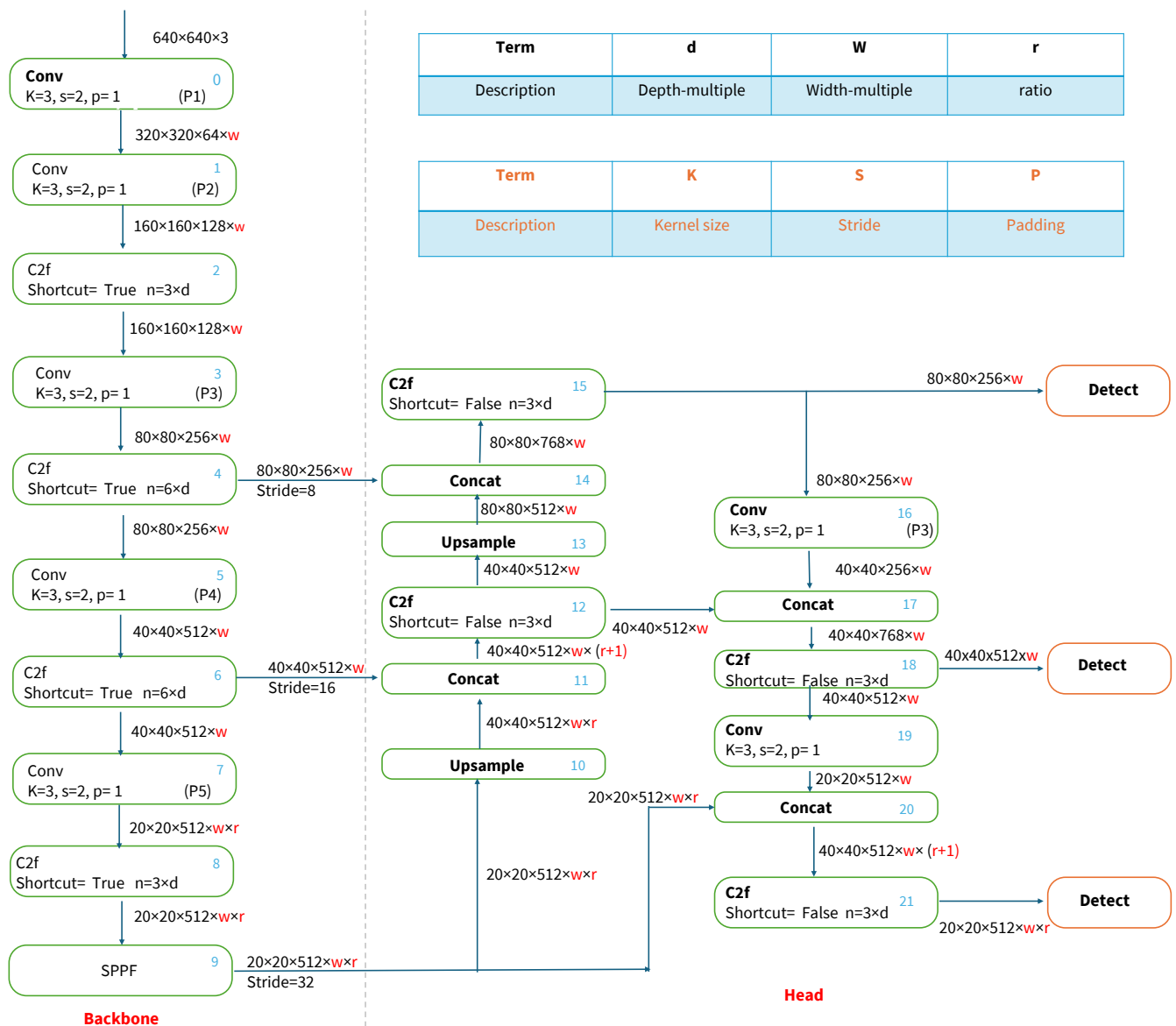


Figure 2. Meta-YOLOv8 architecture, as described in Section 3.1.

The backbone initiates the processing pipeline by progressively extracting discriminative features while reducing spatial dimensions through a sequence of convolution blocks, C2F modules, and SPPF operations [28]. This hierarchical approach preserves both fine-grained signal details and high-level contextual information critical for distinguishing between similar rear-light patterns [29]. The neck module then synthesizes these multi-scale features through cross-layer connections, enabling robust representation of signals across varying distances and orientations. Finally, the detection head leverages these fused features to simultaneously predict signal classifications and precise bounding box coordinates, ensuring accurate localization even for overlapping or partially occluded targets [30].

3.1.1. CBS Layers

The architectural implementation employs CBS layers—a composite sequence comprising Convolution, Batch Normalization, and SiLU (Sigmoid-weighted Linear Unit) activation—as foundational building blocks. In our framework, each CBS operation begins with a 3×3 convolutional kernel to extract spatial features, followed by batch normalization to standardize activation distributions across mini-batches. This normalization computes per-channel means and variances over the input batch, scaling and shifting activations to stabilize gradient dynamics during training. The processed features then pass through a SiLU activation function, which applies a sigmoid-gated linear transformation to introduce non-linearity while preserving gradient continuity.

3.1.2. CBS (Batch Normalization and Pooling)

Batch normalization serves a dual role in this pipeline: it accelerates convergence by mitigating internal covariate shift and regularizes the model by introducing noise through batch-statistic dependencies. By constraining activation magnitudes, this process enhances the network's robustness to input variations—a critical feature for detecting rear signals under diverse illumination and weather conditions. The synergistic combination of CBS components ensures stable feature learning across the hierarchical layers of Meta-YOLOv8, enabling precise signal recognition even in low-data regimes.

3.1.3. CBS (SiLU)

Following initial feature processing through convolution and optional batch normalization, the architecture applies the SiLU (Sigmoid Linear Unit) activation function to introduce non-linear transformations. As shown in Figure 3, the C2F block employs a structured bottleneck design that begins with a 1×1 convolution (stride 1, no padding) to halve channel dimensions and reduce computational complexity [31]. This compressed feature representation is then propagated through two parallel pathways: a double-convolution bottleneck layer and an optional residual shortcut connection. When activated, the shortcut preserves gradient flow by directly linking input and output features, mitigating information loss during deep feature propagation. The outputs from both pathways are concatenated to retain multi-scale contextual information, followed by a final convolutional layer to harmonize feature maps for downstream processing. This dual-path mechanism enhances the model's capacity to balance computational efficiency with feature fidelity, a critical requirement for detecting subtle variations in rear signal patterns under dynamic real-world conditions.

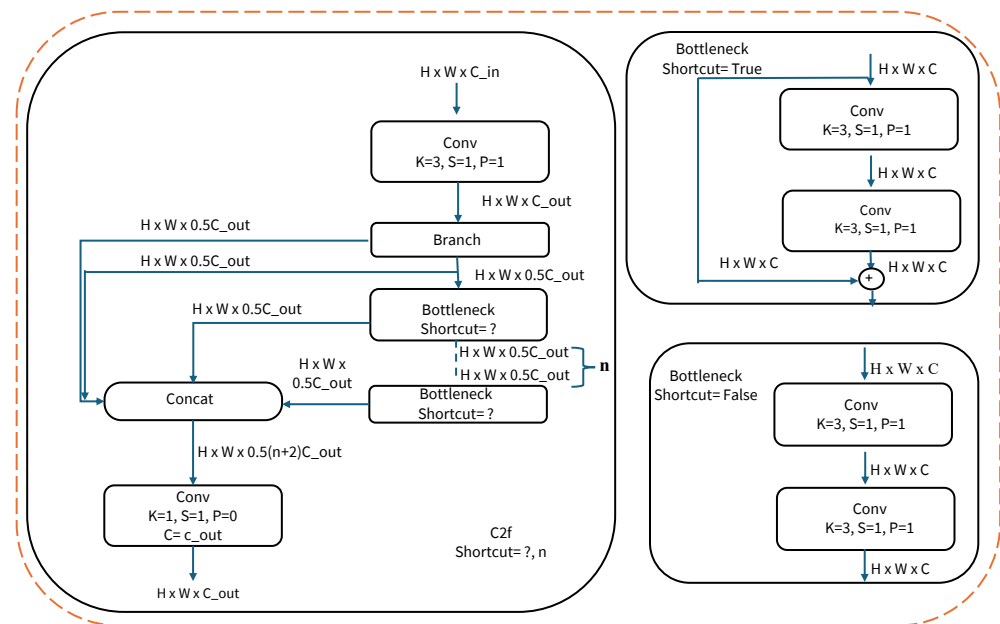


Figure 3. Double-convolutions with cross-stage partial bottleneck (C2F).

3.1.4. Spatial Pyramid Pooling Fast (SPPF)

The Meta-YOLOv8 framework incorporates an optimized variant of spatial pyramid pooling (SPP), termed spatial pyramid pooling fast (SPPF), to balance computational efficiency with multi-scale feature extraction [32]. Figure 4 illustrates how the SPPF module consists of an initial convolutional layer followed by three cascaded max-pooling operations. A key innovation lies in its concatenation of outputs from successive pooling layers, which are then propagated to a final convolution layer for feature refinement. This streamlined design builds on the foundational SPP concept, which partitions input features into hierarchical grids to pool multi-scale contextual information independently across regions (<https://github.com/ultralytics/ultralytics/issues/189>, accessed on 27 October 2025). By aggregating features at varying receptive fields, SPPF enables networks to process objects of diverse scales—a critical capability for detecting rear signals across vehicles of differing sizes or distances.

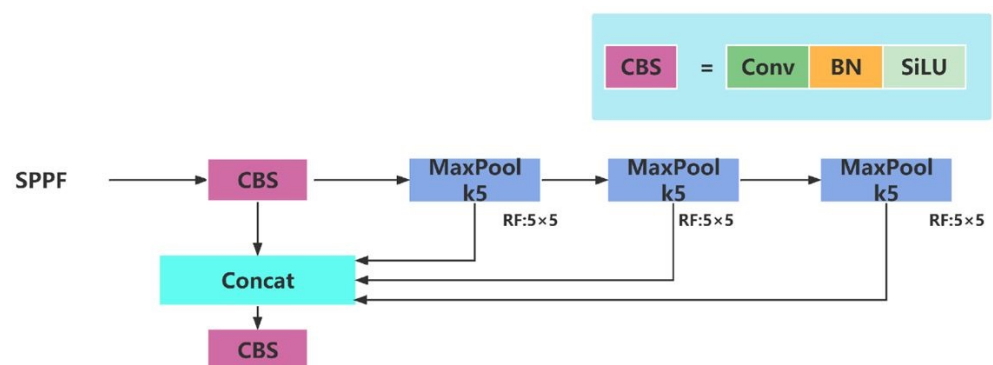


Figure 4. Block diagram of the fast spatial pyramid pooling module.

However, traditional SPP implementations incur high computational costs due to parallel pooling operations with heterogeneous kernel sizes. SPPF addresses this limitation through a sequential pooling strategy using a single fixed kernel size, significantly reducing computational overhead while retaining multi-scale representational capacity. Though this simplification introduces a marginal trade-off in granularity, it preserves detection accuracy

for vehicular signals—which exhibit relatively standardized geometric properties—while enhancing inference speeds crucial for real-time systems. This makes Meta-YOLOv8 more efficient and suitable for devices with limited resources.

3.1.5. Detection Block

The detection block in YOLOv8 is responsible for identifying objects in images [31,32]. Unlike previous versions (at the time of our project), YOLOv8 adopts an anchor-free approach, predicting object centers directly instead of using offsets from predefined anchor boxes. This design enables faster and more efficient predictions. The detection block has two branches: one for bounding box prediction and another for class prediction. Each branch comprises two convolutional blocks followed by a Conv2D layer, as illustrated in Figure 5, which compute the bounding box and class losses, respectively [10].

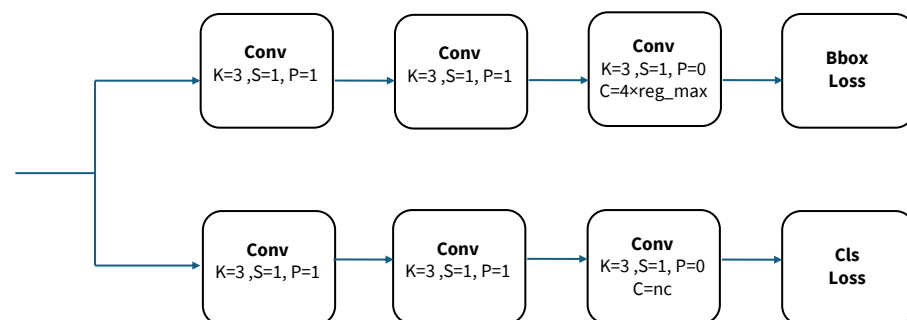


Figure 5. YOLOv8 anchor-free architecture for object center prediction with independent bounding box and class branches.

3.2. Meta-Learner

Our meta-learning framework implements a hierarchical optimization process central to the study’s contributions. The outer loop (Figure 6) iteratively refines base model weights by minimizing a task-agnostic loss function, thereby distilling cross-task feature representations [8,10]. This phase establishes a generalized parameter initialization, observable as a trajectory of decreasing loss values in the weight space.

Subsequently, the inner-loop meta-learner adapts these generalized weights to task-specific objectives through a focused optimization regime (Figure 6b). Leveraging second-order gradient computations (Equation (1)), this stage fine-tunes parameters within a constrained loss landscape tailored to automotive signal detection. The dual-phase architecture—comprising task-similarity learning (outer loop) and task-specific refinement (inner loop)—enables rapid adaptation to new signal patterns while preserving robustness against catastrophic forgetting.

Upon exposure to task-specific data, the model performs final weight updates that specialize its detection capabilities. For instance, in a 3-way 5-shot episode, the model is trained on three classes—Brake Indicating Signal (BIS), a generic Signal (active indicator without side disambiguation), and Car Leaving (CL)—with only five labeled samples per class. The meta-learning inner loop adapts to this small support set, while the outer loop generalizes across many such tasks. This episodic setup illustrates how the model “learns to learn” and adapts rapidly to unseen taillight styles or environmental conditions. In our framework, the post-processing layer (PPL) further disambiguates the generic Signal into left (LIS) or right (RIS) indicators based on geometric orientation. These updates optimize spatial and spectral sensitivity to rear signals, enabling targeted accuracy without sacrificing inference speed. The two-stage optimization framework, therefore, bridges

generalization and specialization, a critical step toward deploying adaptive vision systems in dynamic automotive environments [8,9].

$$\theta' = \arg \min_{\theta} \frac{1}{M} \sum_{i=1}^M \mathcal{L}(\text{in}(\theta, D_i^{\text{tr}}), D_i^{\text{test}}) \quad (1)$$

The individual components in Equation (1) are specified as follows. The term M indicates the number of tasks grouped together, while D_i^{tr} and D_i^{test} correspond to the i^{th} training and testing tasks, respectively. The task loss function is denoted by $\mathcal{L}$, utilizing the data from D_i^{tr} during the inner loop optimization. For each sampled task, the neural network begins with parameters θ . These initial parameters are then updated at the head of Meta-YOLOv8 through one or more steps of gradient descent on D_i^{tr} , producing the adapted parameters θ'_i . Considering only the training phase of the detector, the assignment of parameters is consistent with [8,9].

$$\Theta_i \approx \text{inner}(\theta', D_i^{\text{tr}}) = \theta' - \alpha \nabla_{\theta'} \mathcal{L}(\theta', D_i^{\text{tr}}) \quad (2)$$

This procedure updates the metaparameters θ from Equation (1) by computing the average loss over the fine-tuned parameters Θ_i for each individual task, utilizing the test dataset D_i^{test} . As a result, following fine-tuning, Meta-YOLOv8 achieves more effective loss optimization compared to standard pre-training approaches, as previously discussed. Several modifications enhance learning speed, efficiency, and the model's adaptability to novel tasks and varying task distributions. Further detailed discussions and interactive analyses of these variations are provided in [8,10].

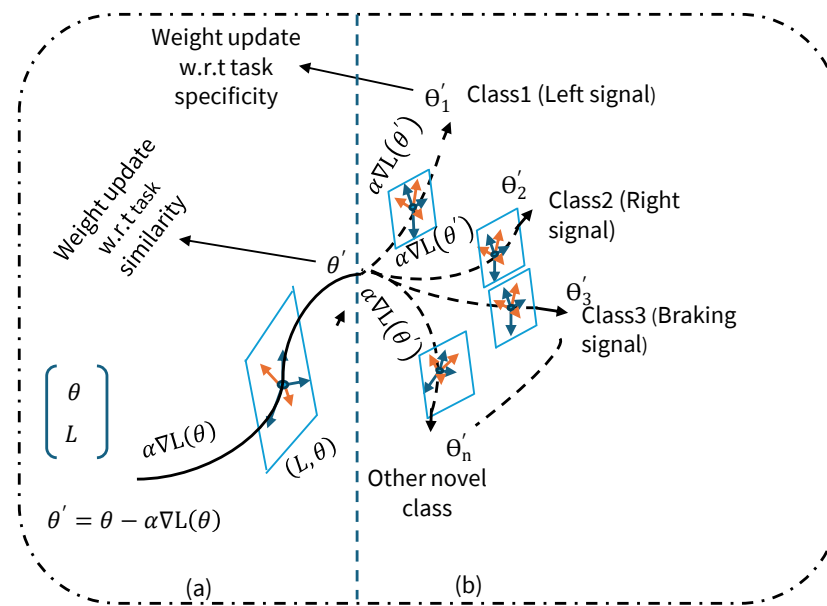


Figure 6. The base model (a) for signal detection starts with random weights θ and is trained on related tasks to prepare it for final task performance, with its learning process driven by a predefined loss function and iterative updates to θ' . Meta-learner (b) then further adjusts these weights to produce Θ' , aligning them with the target task's requirements, before fine-tuning with task-specific data to obtain Θ'_i , optimized for each class detection.

3.3. Post-Processing Layer (PPL)

We introduce a geometric post-processing layer (PPL) that operates on the detections of the meta-trained YOLOv8 model to produce reliable left–right turn assignment. Without PPL, directly classifying LIS vs. RIS is error-prone because the two indicators are visually similar,

small, and often imbalanced; yet, side assignment is essential for interpreting the lead vehicle's intent in ADAS/AV decision-making. At inference, the detector gives three labels: *Car Leaving (CL)*, *Brake Indicating Signal (BIS)*, and *Signal* (generic active indicator). The PPL then applies lightweight, deterministic geometry to each *Signal*: (i) associate it to the corresponding CL box via an IoU check; (ii) compute the CL horizontal midpoint and the *Signal* center to perform side assignment (LIS/RIS); and (iii) apply edge/occlusion corrections using an area-ratio heuristic and optional optical-flow consistency. When no reliable CL is available, the system issues an unassigned *Signal* alert.

This post-detection stage—summarized in Figure 7 and formalized in the corresponding equations—consistently upgrades raw detections into intent-level outputs (LIS/RIS) with minimal runtime overhead.

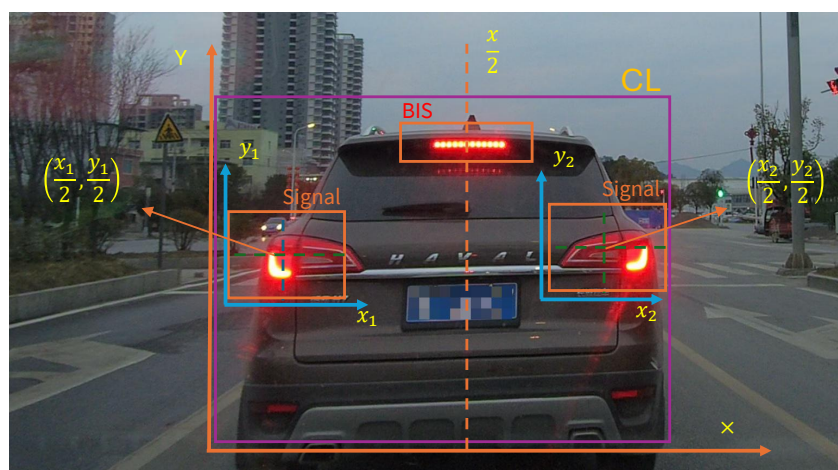


Figure 7. Visual Representation of PPL-Enhanced Signal Classification. BIS denotes the brake signal, and *Signal* denotes a generic active turn indicator prior to side assignment (LIS/RIS).

Mathematically, the logic illustrated in Figures 7 and 8 can be formalized as follows. Given the bounding box of the detected Car Leaving (CL), $B_{CL} = (X_1, Y_1, X_2, Y_2)$ its horizontal midpoint is $m_{CL} = \frac{X_1 + X_2}{2}$.

For each detected signal box $B_{s_i} = (x_1^i, y_1^i, x_2^i, y_2^i)$, $i = 1, \dots, n$, the signal bounding box (Bbox) center is $c_{s_i} = (\frac{x_1^i + x_2^i}{2}, \frac{y_1^i + y_2^i}{2})$.

The left/right assignment rule shown in Figure 7 is

$$\text{cls}(s_i) = \begin{cases} \text{LIS}, & \text{if } c_{s_i} < m_{CL}, \\ \text{RIS}, & \text{if } c_{s_i} > m_{CL}, \end{cases} \quad i = 1, \dots, n. \quad (3)$$

Finally, to ensure that s_i truly belongs to the detected car, we compute

$$\text{IoU}(B_{CL}, B_{s_i}) = \frac{|B_{CL} \cap B_{s_i}|}{|B_{CL} \cup B_{s_i}|}. \quad (4)$$

If $\text{IoU}(B_{CL}, B_{s_i}) > 0.5$, the signal s_i is associated with B_{CL} .

3.4. Meta-Learning for Efficient Adaptation in Meta-YOLOv8 Training

The Meta-YOLOv8 training approach differs from conventional YOLOv8 by employing meta-learning to adapt model weights using data from tasks related to the target domain. In meta-learning, a model acquires general strategies that enable rapid adaptation to new tasks with minimal data, rather than optimizing solely for a single task. This capability allows Meta-YOLOv8 to generalize better and fine-tune more efficiently in low-data or dynamically changing environments while preserving YOLOv8's strengths in real-time object detection. Details of the episodic setup and datasets are provided in Section 4.

Such adaptability is especially valuable when data scarcity or task variability limits conventional training. For example, in *car rear signal detection*—identifying brake lights and turn signals—collecting a large, diverse dataset is challenging. These signals are often small, vary in shape and color between car models, and appear under different lighting conditions, making data collection and annotation labor-intensive. Meta-learning addresses these issues by leveraging shared characteristics of vehicle signals, such as their typical placement and consistent chromatic patterns. By exploiting these similarities, Meta-YOLOv8 can generalize effectively from limited samples, reducing overfitting and mitigating forgetting during rapid adaptation—challenges that often hinder traditional deep learning methods.

4. Experimental Setup and Methodology

This section outlines the experimental framework, detailing the dataset composition, preprocessing methodology, and annotation protocols. We specify the computational architecture's configuration and define the evaluation metrics employed to quantify detection performance across all trials.

4.1. Data

Through a multi-criteria framework, we synthesized a fusion dataset (<https://github.com/VasuTammisetti/Real-Time-Camera-Based-Rear-Car-Signal-Detection-in-AD-AS-Using-Meta-Learning/tree/main/data>, accessed on 27 October 2025) from diverse public repositories such as KITTI (https://www.cvlibs.net/datasets/kitti/eval_object.php?obj_benchmark=2d, accessed on 27 October 2025), CARLA (<https://www.kaggle.com/datasets/sachsene/carla-traffic-lights-images>, accessed on 27 October 2025), LISA (<https://www.kaggle.com/datasets/mbornoe/lisa-traffic-light-dataset/code>, accessed on 27 October 2025), City Scapes (<https://www.cityscapes-dataset.com/login/>, accessed on 27 October 2025), Eurocity (https://eurocity-dataset.tudelft.nl/eval/user/login?_next=/eval/downloads/detection, accessed on 27 October 2025). Table 1 presents a summary of these.

We prioritized high-resolution images captured under varying illumination, weather conditions, and diverse vehicle orientations, including lateral and driver perspectives. We also focused on several key aspects: the morphology of signal lights (accounting for size and shape variations across vehicle brands); optimizing resolution to ensure object clarity; maintaining aspect-ratio consistency to preserve geometric fidelity; and preserving the spectral integrity of signal hues, enabling robust classification even under challenging environmental conditions.

Table 1. Used datasets with features of varying significance, coupled with varying degrees of data accuracy and integrity.

Sno	Dataset	Main Components	Reliability/Noise in Data
1	KITTI	Brake signals and the car's rear view, car signal lights.	90%/10%
2	CARLA Images	Color of brake signals and the car's rear view, car left and right signal lights weather conditions.	85%/20%
3	LISA Dataset	Brake, left and right signals, and the car's rear view, weather conditions, and traffic scenes at junctions.	80%/20%
4	Cityscapes	Brake, left and right signals, and the car's rear view.	85%/15%
5	Eurocity	Brake, left and right signals, and the car's rear view at different junction points.	90%/15%

Images were curated to replicate operational sightliness at driver-relevant distances (approximately 5–50 m) while incorporating edge-enhanced features to strengthen contour detection in occluded scenarios. The dataset explicitly includes complex urban contexts—intersections with multi-vehicle interactions and overlapping signals—to stress-test model performance under real-world ambiguity. Preprocessing enforced perspective alignment through homography adjustments and adaptive histogram equalization to mitigate lighting artifacts. This structured approach ensures the model trains on edge cases mirroring autonomous systems' perceptual challenges while maintaining detection precision across a dynamic automotive environment. Noise percentages in Table 1 reflect overall dataset quality, not task-specific errors. Since these datasets were collected for broader purposes, some frames included occlusions, glare, or unrelated scenes. For our experiments, only task-relevant samples were hand-picked to ensure suitability for rear-signal detection.

For each experiment described in Section 5, we further partitioned the dataset into three subsets tailored to distinct evaluation criteria: task adaptability (utilized in Sections 5.2–5.4), task similarity (used in Sections 5.1 and 5.4), and task specificity (also used in Sections 5.1 and 5.4). The task-similarity dataset consists of 319 images (<https://zenodo.org/records/13969232>, accessed on 27 October 2025) featuring traffic signals, encompassing approximately 1500 instances of red, green, and orange signals combined. For task specificity, we employed a dataset containing 115 images, which was expanded to 180 images through augmentation, capturing around 300 instances of brake signals and 650 instances of rear car turn-indicator signals. Lastly, the task-adaptability dataset comprises 40 images, including 35 instances of brake signals and 85 instances of rear car signals. All datasets are publicly available through the links provided.

4.2. Data Preprocessing

The preparation of data is always essential for achieving accuracy and efficiency [33–35]; thus, we applied the following preprocessing steps to the raw dataset described earlier:

Data cleaning: We removed damaged or unsuitable images, including those that were blurred, poorly exposed, or lacked any visible car rear signal lights. This ensures that the dataset contains only high-quality images that are important to the task accuracy.

Image resizing: To ensure consistent alignment with the training model and alleviate computational demands, we resized all images to a standard dimension while preserving their aspect ratio. This uniformity is key to efficient batch processing during model training.

Normalization: Pixel intensities were standardized to a zero mean and unit variance. This process promotes faster model convergence during training and boosts its ability to generalize. Each pixel intensity $I_{norm}(x, y)$ was normalized as $I_{norm}(x, y) = \frac{I(x, y) - \mu}{\sigma}$, where μ and σ are the mean and standard deviation of the image.

Augmentation: To increase the dataset size, we applied data augmentation techniques such as random rotations, flipping, scaling, and cropping. This approach helps prevent overfitting and enhances the model's robustness to typical real-world variations, such as changes in the angles and sizes of a car's rear signal lights.

Color space conversion: Images were converted to the HSV (Hue, Saturation, Value) color space, which separates color information from brightness, making it easier to highlight signal lights under varying lighting conditions. This enhances detection robustness by maintaining a consistent color representation despite changes in illumination.

Contrast adjustment: To improve the visibility of tail indicator lamps under dim lighting, we employed histogram equalization to dynamically enhance image contrast. This technique sharpens the distinction of signal flashes, enabling the model to more reliably detect them across diverse and challenging environmental conditions. The enhanced intensity $\hat{I}(x, y)$ was obtained as $\hat{I}(x, y) = \frac{L-1}{N} \sum_{k=0}^{I(x, y)} h(k)$, where L is the number of intensity levels, N is the number of pixels, and $h(k)$ is the histogram count at level k .

Noise reduction: To enhance image quality, noise suppression methods such as Gaussian blur and median filter were applied. These techniques smooth the images by minimizing sensor noise and compression artifacts, thereby improving the clarity of the signal lights.

Edge enhancement: Edge detection filters, such as Sobel and Canny, were applied to highlight the contours of car's rear signal lights. This processing helps the model distinguish these signals from cluttered environments, thereby improving recognition accuracy. The gradient magnitude was computed as $G(x, y) = \sqrt{(G_x(x, y))^2 + (G_y(x, y))^2}$.

4.3. Task Generation

Our meta-learning framework adopts a task-oriented training paradigm to ensure balanced class representation during training. While earlier techniques addressed dataset-level class imbalance, this framework maintains uniform class distribution at the task level. It constructs training episodes using systematically sampled 3-way, 3/5/8/10-shot tasks, with a data pipeline assembling balanced image-label pairs for every task. This episodic strategy minimizes residual bias toward frequent classes, ensuring fairness during both base model initialization and task-specific adaptation [8,10].

By decoupling training from stochastic batch statistics, the framework stabilizes gradient updates across both rare and common signal categories—a critical requirement for maintaining detection consistency in low-data regimes. This helps the model learn general features while staying ready to specialize for each task, essential for handling signal variability in real-world automotive environments without inducing catastrophic forgetting.

4.4. Evaluation Metrics

Several key evaluation metrics—including precision, recall, F1-score, Intersection over Union (IoU), and mean Average Precision (mAP)—were utilized to comprehensively assess the models' detection accuracy and localization quality across multiple object categories. In addition to these performance indicators, the models' processing speeds, measured in frames per second (FPS), were analyzed to gauge their suitability for real-time applications. The evaluation also encompassed robustness testing under diverse environmental condi-

tions, such as variable lighting and weather scenarios, as well as assessments of detection range and occlusion handling, to verify each model's reliability and practical effectiveness in real-world settings.

Given true positives (TP), false positives (FP), and false negatives (FN), the following metrics can be defined:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (8)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (9)$$

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (10)$$

$$\text{IoU} = \frac{|B_{pred} \cap B_{gt}|}{|B_{pred} \cup B_{gt}|} \quad (11)$$

where B_{pred} and B_{gt} are predicted and ground-truth bounding boxes.

Average Precision (AP) is defined as the area under the precision–recall curve:

$$AP = \int_0^1 p(r) dr \quad (12)$$

where $p(r)$ denotes precision as a function of recall. The mean AP across all classes is

$$mAP = \frac{1}{C} \sum_{i=1}^C AP_i \quad (13)$$

where C is the number of classes.

4.5. Experiment Setup

Our setup integrates both hardware and software components to support the training and deployment phases of the car rear signal detection model. For training purposes, we leveraged Tesla T4 and A100 GPUs accessible via the Google Colab environment. Inference on edge devices was conducted using the NVIDIA Jetson Nano platform. Dataset preparation involved manual annotation of images with the assistance of LabelMe and Makesense AI, which are specialized image labeling tools.

The overall model development was performed in Python 3.10, utilizing TensorFlow (version 2.8.0) alongside PyTorch (version 2.2.1). Image preprocessing, including transformations and feature extraction, was handled through OpenCV (version 4.8.1), while Matplotlib (version 3.8.0) was employed to generate visualizations for data analysis.

As depicted in Figure 9, the workflow begins with a base YOLOv8 operating on task-relevant inputs. The green module performs episodic meta-training with explicit weight sharing across tasks, yielding a meta-initialized YOLOv8 in line with MAML-style optimization [8,10]. During inference, the resulting detections are refined by the post-processing layer (PPL) to produce final rear-signal intents.

All the code, experiments, and additional resources can be found in the paper's code repository (<https://github.com/VasuTammiseti/Real-Time-Camera-Based-Rear-Car-Signal-Detection-in-ADAS-Using-Meta-Learning>, accessed on 27 October 2025).

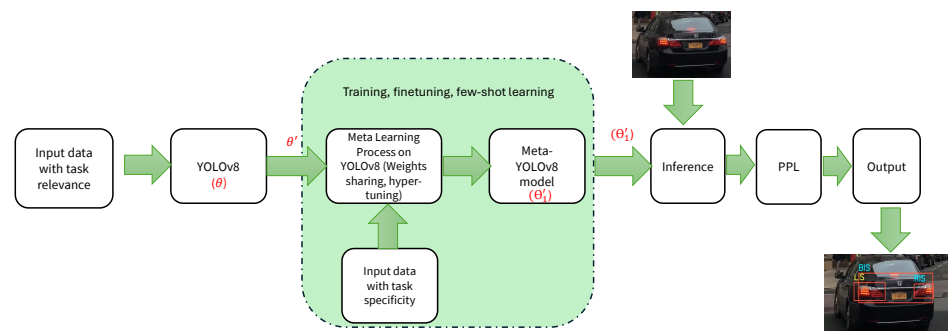


Figure 9. Schematic representation of the data processing pipeline in Meta-YOLOv8.

4.6. Experimenting with Training Methodologies

In the initial training stage, we employed a relatively high learning rate (0.1) together with a substantial momentum term to promote the extraction of high-level visual features from input images. In the subsequent phase, a systematic hyperparameter optimization process was carried out using the AutoKeras framework [36], through which we identified a learning rate of 0.0085 and a momentum of 0.935 as the most effective settings. Guided by the concept of task similarity, these optimized values proved instrumental in improving car rear-signal classification during pre-training. Within the Meta-YOLOv8 training process, model weights are strategically initialized and progressively refined from the outset, as illustrated in Figure 6, enabling the transfer and utilization of prior knowledge from related domains. For instance, pre-training on datasets such as vehicle lights or traffic signals—sharing visual and structural similarities with rear-signal lights—provides a strong representational foundation for light detection and classification. Building upon this foundation, the model undergoes incremental fine-tuning on smaller, task-specific subsets of rear-signal data, with each step further enhancing detection accuracy and robustness. This iterative refinement leads the model to converge toward optimal parameters for rear-signal detection, even under challenging conditions such as poor visibility or occlusions (see Figure 10a,d). It is also crucial to distinguish between meta-learning and conventional fine-tuning: while fine-tuning adapts a pre-trained model through extended training on task-specific data, meta-learning equips the model with the inherent ability to adapt efficiently to new tasks with minimal retraining [8].

To develop a robust model for accurate vehicular signal classification, we conducted controlled experiments employing multiple training strategies to address the significant challenge posed by the visual similarity between left and right turn signals. Despite the brake signal being consistently recognized with high accuracy due to its unique visual properties, left and right turn signals were frequently misclassified, primarily due to their similarities in color, shape, and spatial configuration. To mitigate this, we explored various labeling schemes, feature extraction techniques, and architectural enhancements aimed at increasing the discriminative power of the model.

We experimented with state-of-the-art object detection architectures, including Single Shot MultiBox Detector (SSD), Detection Transformers (DETR), and You Only Look Once version 8 (YOLOv8). Additionally, we implemented ensemble techniques, combining YOLOv8 with both standard and meta-learning-trained models. However, these approaches only yielded marginal improvements of 1–2%, highlighting that no single model sufficiently distinguished left and right signals purely from visual features.

The PPL (explained in Section 3.3) leveraged spatial relationships to further classify signals based on their relative positions. Specifically, we trained the meta-learning model to output three distinct classes: brake signal, signal (a combined category for both left and right turn signals), and the rear-car bounding box (CL) used as a geometric reference. In the final implementation, our model generated annotated images or videos indicating detections of

signal, brake signal, and Car Leaving. The post-processing layer then utilized these outputs to classify the signals based on their spatial positioning relative to the bounding box of the Car Leaving. This innovative spatial context integration significantly improved the model's reliability in distinguishing left and right signals, underscoring the importance of advanced processing techniques for accurate classification (see Figure 10).



Figure 10. Refined detection and classification of car rear signals using Meta-YOLOv8 with PPL. Subfigures (a,d) demonstrate robust performance under glare and partial occlusion, showing that the model maintains accurate signal detection even in challenging illumination. (b,c) Illustrate adaptability to variations in taillight design and shape. (LIS = Left Indicator Signal; RIS = Right Indicator Signal; BIS = Brake Indicating Signal).

To demonstrate the performance of our approach, we applied meta-learning techniques in training YOLOv8 for car rear signal detection with a limited dataset comprising 115 images. Our goal was to achieve robust real-time detection and classification capabilities for brake lights and turn signals, even with novel or previously unseen vehicle models, as shown in Figures 10b and 11b. Leveraging meta-learning allowed the model to effectively generalize from a minimal set of training examples, transferring knowledge gained from related tasks such as general vehicle light recognition or traffic signal detection [8,10]. This significantly mitigated challenges associated with limited data availability, enhancing adaptability across various scenarios, including different car signal models and adverse lighting conditions such as nighttime or heavy rain.

Overall, the Meta-YOLOv8 framework with PPL provides an effective solution for car rear signal detection in Advanced Driver Assistance Systems (ADAS) by strategically initializing and incrementally refining the model's weights, effectively addressing data scarcity and enhancing generalization. The key advantage of meta-learning, “learning to learn better”, facilitates swift adjustment to novel tasks despite having minimal data available, unlike traditional fine-tuning methods that require extensive retraining. Consequently, our methodology is particularly suited to addressing variability in rear signal designs across diverse vehicle manufacturers, demonstrating notable accuracy, robustness, and adaptability.

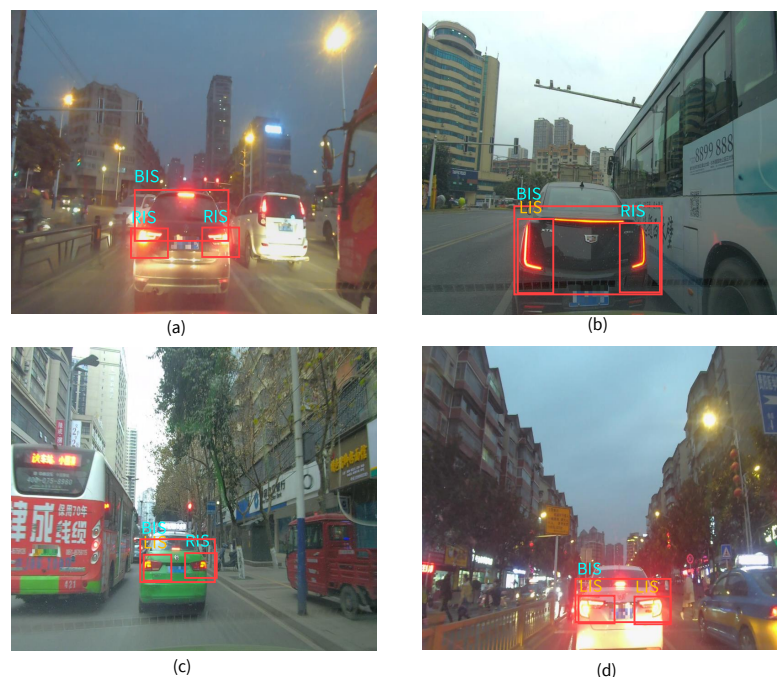


Figure 11. Refined detection and classification of car rear signals using only Meta-YOLOv8 without the PPL. Subfigures (a,d) show cases of misclassification caused by the absence of geometric post-processing: in (a) both indicators are incorrectly labeled as right (RIS), and in (d) both are classified as left (LIS). (b,c) Demonstrate correct identification and adaptability of the model to new taillight shapes and designs. (LIS = Left Indicator Signal; RIS = Right Indicator Signal; BIS = Brake Indicating Signal).

5. Results and Discussion

In this section, we compare the performance of Meta-YOLOv8 with a post-processing layer (PPL) against both the conventional Meta-YOLOv8 ensemble approach and the standalone YOLOv8 model. Emphasis is placed on scenarios with scarce or highly specific datasets, where robust generalization is critical. We focus on key performance indicators such as F1, mAP, and precision, supported by visualizations that illustrate the improved recognition capability and adaptability of Meta-YOLOv8 across diverse conditions.

In the multi-class YOLOv8 framework, each bounding box is associated with an objectness score and a set of independent class probabilities obtained via sigmoid activations. Among evaluation metrics, we highlight the F1-score because it provides a single harmonic measure that balances precision and recall. This is particularly important in taillight detection, where adjusting the confidence threshold strongly influences the precision–recall trade-off: lower thresholds increase recall but risk more false positives, while higher thresholds improve precision but reduce recall. The F1-score captures this balance, making it a natural choice for identifying the optimal operating threshold.

The overall detection confidence is defined as

$$\text{Confidence (F1)} = P(\text{object}) \times \max_{c \in \{1, \dots, C\}} P(\text{class} = c) \quad (14)$$

where C denotes the number of classes (Brake, LIS, RIS, CL, Signal). By sweeping the decision threshold on this scalar confidence value from 0 to 1, we obtain different precision–recall trade-offs. The F1-score–Confidence curve thus illustrates how the balance between precision and recall varies with the threshold, with its peak representing the most reliable operating point.

5.1. Comparative Analysis Between Meta-YOLOv8 and the Standard YOLOv8 Model

Initially, we experimented with direct classification of left and right signals without employing a post-processing layer (PPL). However, this approach yielded suboptimal results due to the high similarity between the two classes, which led to strong bias and poor classification accuracy (Figure 12c,d). To reduce task complexity, we next grouped left and right signals into a single “signal” class. This strategy improved model stability and overall performance (Figure 12b), but it still lacked the reliable classification required for robust deployment.

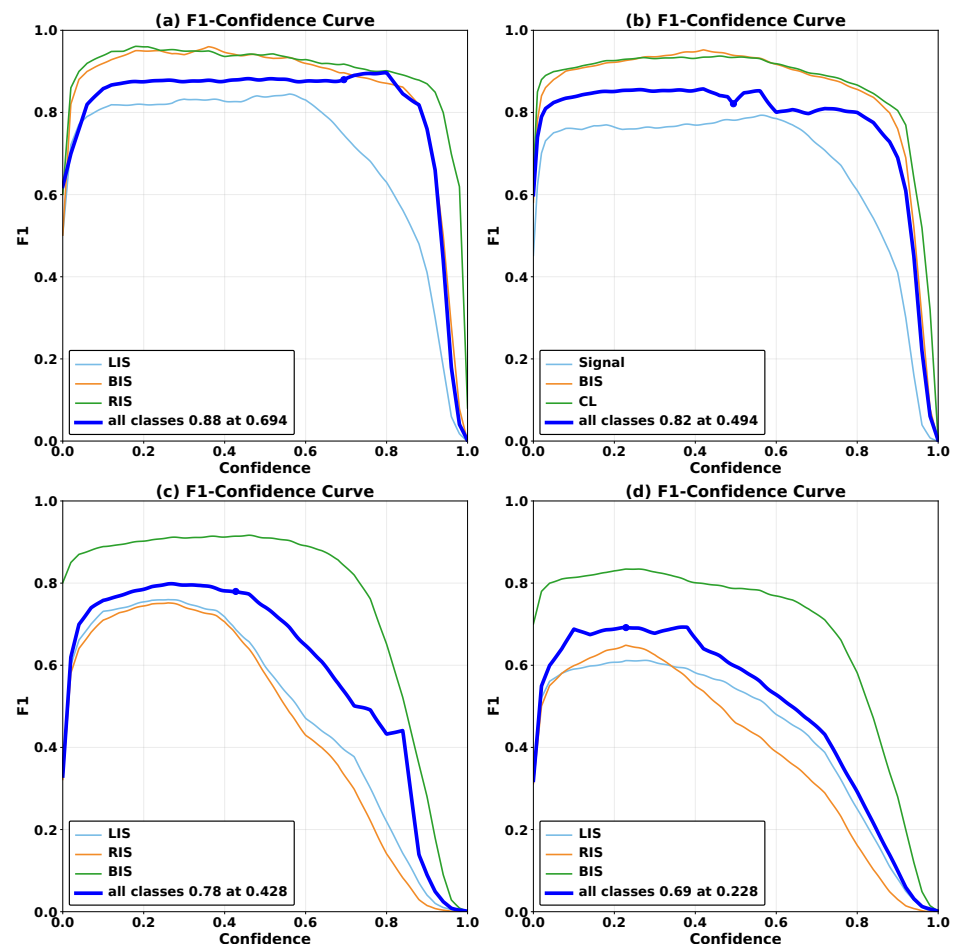


Figure 12. F1 scores for signal classification comparing four approaches: (a) Meta-trained YOLOv8 with post-processing layer (PPL); (b) Meta-trained YOLOv8 without PPL (left and right signals combined); (c) Meta-trained YOLOv8 without PPL (direct separate classification); (d) Conventionally trained YOLOv8 (direct separate classification). (LIS: Left Indicating Signal, RIS: Right Indicating Signal, BIS: Brake Indicating Signal, CL = rear-car bounding box).

To achieve accurate and reliable disambiguation of left and right signals, we integrated a post-processing layer (PPL) on top of the Meta-YOLOv8 framework (Section 3.3). As shown in Figures 10a,b, 11a,b and 12a,c, this refinement corrected many misclassifications and raised the detection confidence to 88%. Even the simplified approach that merged left and right signals into a single class achieved 82% confidence (Figure 12b), confirming that this intermediate strategy mitigates some of the challenges of direct classification (Figure 12c,d). Nevertheless, the Meta-YOLOv8+PPL framework consistently provided the most robust and interpretable results.

A comparative analysis of the performance indicators among the tested models demonstrates notable disparities in effectiveness. The Meta-YOLOv8+PPL configuration showed

better results, attaining a peak precision of 89% (see Figure 13a), whereas the baseline YOLOv8 achieves only 66% precision (refer to Figure 13d). Key performance metrics include mean Average Precision at 50% IoU (mAP@50) and the average mAP over IoU thresholds from 0.5 to 0.95 (mAP@0.5:0.95), further highlighting the superiority of the Meta-YOLOv8 models. Specifically, the Meta-YOLOv8+PPL model achieves a mAP@50 of 89% and a mAP@0.5:0.95 of 69.4%, surpassing the baseline YOLOv8 model (mAP@50 of 65.8%), the Meta-YOLOv8 (Left and Right Combined as Signal) Figure 13b model (mAP@50 of 84.9%), and the Meta-YOLOv8 (Direct Classification of Left, Right, and Brake Signals) model (mAP@50 of 78%; Figure 13c). These results highlight the improved localization and classification performance of the Meta-YOLOv8 models, particularly when augmented with PPL. In particular, the mAP@0.5:0.95 metric demonstrates the model's robustness under challenging detection scenarios, including adverse weather, glare, and high frame complexity, see Figure 14.

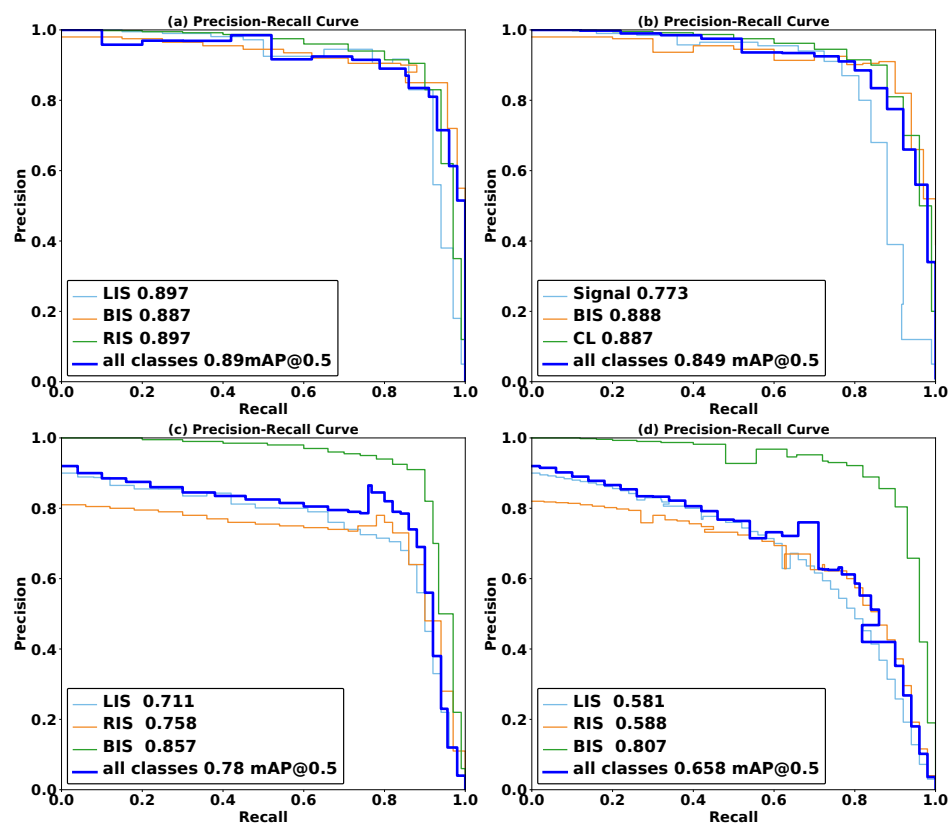


Figure 13. Precision and recall curves comparing four signal classification approaches: (a) Meta-trained YOLOv8 with post-processing layer (PPL); (b) Meta-trained YOLOv8 without PPL (left and right signals combined as signal); (c) Meta-trained YOLOv8 without PPL (direct classification); (d) Conventionally trained YOLOv8 (direct classification). (LIS: Left Indicating Signal, RIS: Right Indicating Signal, BIS: Brake Indicating Signal, CL = rear-car bounding box).

Integrating the PPL not only improved precision and confidence but also strengthened the model's generalization and adaptability to dataset variations. This adaptability is crucial for real-world applications, where maintaining high performance under diverse conditions and designs is essential (see Figure 10). The Meta-YOLOv8+PPL model achieves these results using limited training data, demonstrating its efficiency and its potential to reduce computational demands during development and deployment (while it requires slightly more processing than Meta-YOLOv8 due to the added post-processing layer, its resource usage remains lower than that of conventional models). The precision rate (PR) of the models further emphasizes the advantage of Meta-YOLOv8+PPL, which achieves high

precision values, indicating accurate identification of relevant instances while minimizing false positives. In contrast, the baseline YOLOv8 model shows lower precision and recall values, resulting in suboptimal performance (see Figure 13). Precision is particularly critical in scenarios where false positives can be costly, such as autonomous driving or safety-critical systems, making the Meta-YOLOv8 models better suited for such applications.

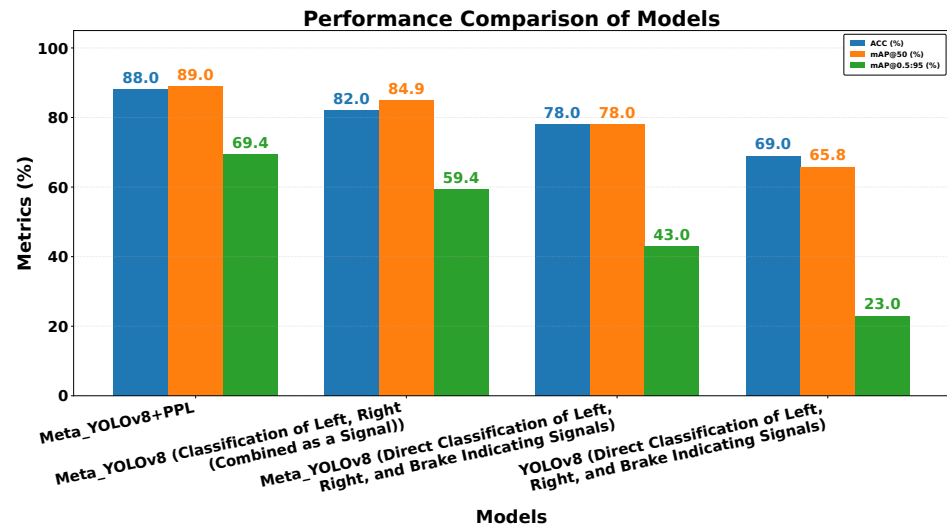


Figure 14. mAP and an accuracy comparison of different signal detection models.

The strong performance of the Meta-YOLOv8+PPL model is driven by its meta-learning capabilities, which improve its ability to recognize patterns in the input data and adapt effectively to new tasks or conditions. This is demonstrated by its capacity to achieve high performance metrics with limited training data, making it particularly effective in scenarios where data availability is constrained. In contrast, the baseline YOLOv8 model lacks these advanced meta-learning features, resulting in inferior adaptability and generalization performance. Additionally, the Meta-YOLOv8 framework offers significant efficiency advantages during both development and deployment. Its ability to deliver better results with reduced computational and data requirements underscores its practical value in resource-constrained environments, while also simplifying post-deployment maintenance. This combination of efficiency and adaptability to new data distributions and operating conditions makes the Meta-YOLOv8 models a robust solution for signal detection under real-world conditions.

The comparative analysis confirms the efficacy of the Meta-YOLOv8 models, particularly the Meta-YOLOv8+PPL variant, in terms of accuracy, precision, and resilience to input data variations. Their ability to achieve high performance with smaller datasets underscores their potential for real-world deployment, where computational efficiency and accuracy are critical. The results demonstrate the effectiveness of meta-learning and PPL as advanced solutions for object detection tasks, as evidenced in Figures 10 and 15. Notably, the model excels even with minimal data, outperforming conventional approaches. This superiority is validated through metrics such as the F1 score and precision–recall curves, which highlight its ability to accurately identify relevant features, see Figure 16, and maintain robust performance across diverse scenarios [37].



Figure 15. Performance and robustness of the Meta-YOLOv8 model with PPL in LIS and RIS detection. Subfigures (a,d) show accurate detections under low-light conditions, while (c) demonstrates stable performance despite strong night glare. In (b), the model correctly detects the active left indicator of one vehicle and the partially visible left signal of another at the frame edge. (LIS = Left Indicator Signal; RIS = Right Indicator Signal; BIS = Brake Indicating Signal).



Figure 16. Performance and robustness of the Meta-YOLOv8 model with PPL in LIS and RIS detection. Subfigures (a) show reliable detection during low-light with multiple vehicles, (b) highlight active signal detection in low-light with a partially visible vehicle, and (c,d) demonstrate strong adaptability under low-light conditions. (LIS = Left Indicator Signal; RIS = Right Indicator Signal; BIS = Brake Indicating Signal).

5.2. Adaptability of the Model

To evaluate the model's adaptability—a key principle of meta-learning—we tested the meta-trained YOLOv8+PPL framework under three adverse environmental conditions that typically degrade object detection performance: rain, fog, and nighttime driving. These conditions were intentionally excluded from the initial training dataset to rigorously assess the model's capacity for rapid adaptation and generalization in unfamiliar and complex environments. Testing was conducted on a Jetson Nano to evaluate the edge compatibility [38] of the model and the post-processing layer (PPL), despite the model being trained on a T4 GPU. This experimental setup was intended to replicate real-world, resource-constrained deployment scenarios to assess the model's robustness and operational efficiency.

A compact collection of 40 images was assembled, with a division of 10 images reserved for training and 3 for validation across each scenario. Leveraging few-shot learning—a main aspect of meta-learning—we conducted experiments to evaluate the model's performance using minimal data. Figure 17 illustrates the model's evaluation on unseen images, with corresponding results detailed in Figures 18 and 19. In low-light and faint visibility conditions, as shown in Figure 18a,b,e, the model successfully detected and classified LIS and RIS. However, in cases where the confidence level of CL was insufficient to classify LIS or RIS explicitly, the model still detected the presence of signals and provided an alert indication, as seen in Figure 18c,d. This demonstrates the model's ability to maintain functionality even in challenging scenarios where the PPL—dependent on CL labels—fails to classify signals explicitly.

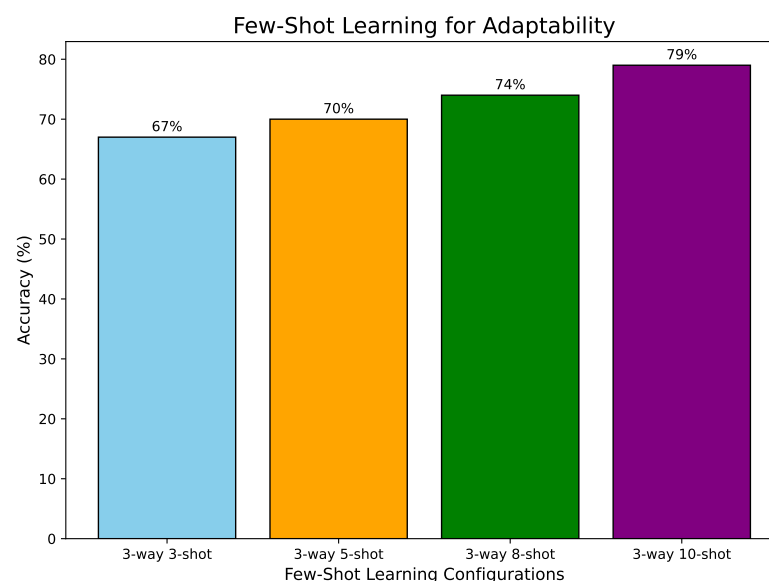


Figure 17. Few-shot learning performance under varying weather conditions.

The model exhibited robust performance under adverse weather conditions, achieving precise detections despite significantly reduced visibility. In rainy conditions, the model demonstrated reliable detection capabilities at distances of approximately 25–30 m, overcoming challenges posed by low visibility. In both heavy and moderate fog, as shown in Figure 19b,c,f, the model maintained stable performance despite visibility dropping to 25% of normal daytime levels, accurately detecting signals at distances of up to 40–48 m. Furthermore, the model efficiently detected and classified signals within an arc angle of 75 degrees, where the center axis aligns with the camera's focal point and the direction of motion. These results underscore the Meta-YOLOv8+PPL model's ability to adapt to novel environments employing limited data. This showcases its strengths in continuous learning

and real-time response, and makes it well-suited for deployment in complex, real-world scenarios with highly variable conditions.

In summary, the experiments confirm the Meta-YOLOv8+PPL model's strong adaptability and edge compatibility, even under challenging and unseen weather conditions. Its ability to provide accurate detections and alert indications in low-visibility scenarios, such as rain, fog, and nighttime driving, underscores its potential for deployment in real-world applications, particularly in Advanced Driver Assistance Systems (ADAS) to improve situational awareness. This adaptability and the model's efficiency make it a promising solution for edge-based signal detection tasks in dynamic and resource-constrained environments.



Figure 18. Adaptability of the Meta-YOLOv8-PPL framework in low-light and glare conditions (tested on Jetson Nano). Subfigures (a,b) show detections under strong night glare, while (c,d) illustrates a case where the car-leaving (CL) box is missing, leading the model to output a generic Signal alert without side classification. Subfigures (e,f) represent similar scenarios with vehicles in very low light, and (f) demonstrates robust detection performance under foggy conditions. (LIS = Left Indicator Signal; RIS = Right Indicator Signal; BIS = Brake Indicating Signal).

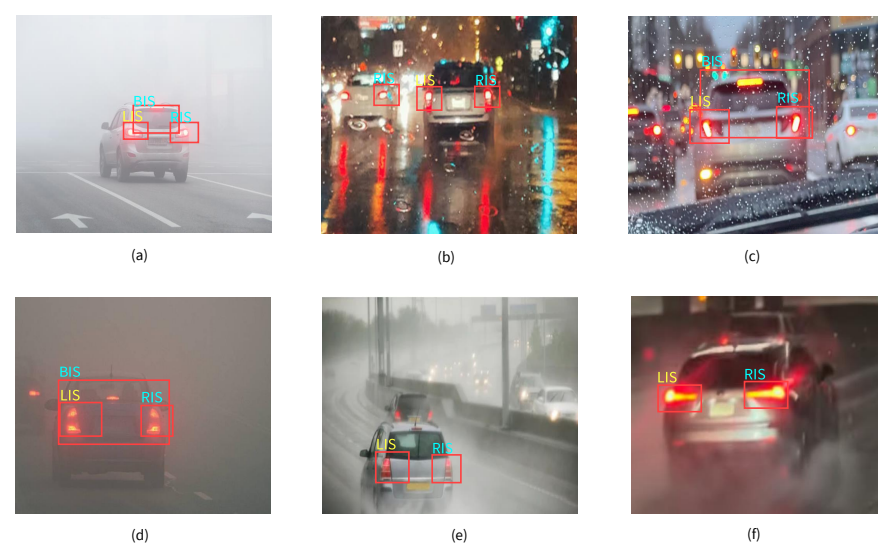


Figure 19. Adaptability of the Meta-YOLOv8-PPL framework under low-light, glare, rain, and fog conditions. Subfigures (a,d) show successful detections in dense fog (tested on Jetson Nano), while (b,c,e,f) illustrate model performance under varying rainfall intensities, ranging from light drizzle to heavy rain. (LIS = Left Indicator Signal; RIS = Right Indicator Signal; BIS = Brake Indicating Signal).

5.3. Performance Analysis of Other Methods

In this section, we discuss two complementary evaluations: a comparative analysis of individual detectors—SSD, YOLOv8, and DETR—under meta and conventional configurations, and the effect of forming ensembles exclusively from meta models. As summarized in Figure 20, meta variants consistently surpass their conventional variants; among single models, Meta-YOLOv8 attains the highest accuracy (78% without PPL), while metamodel ensembles provide only marginal gains (e.g., up to 79%), indicating that ensemble performance is largely bounded by the strongest constituent and remains below Meta-YOLOv8+PPL. We then examine runtime throughput in Figure 21 across edge and cloud hardware (PC i5/4 GB, Jetson Nano, MSI GP65 i7/RTX 2070, and T4 GPU), before and after adaptability training and Flask–Docker deployment. Continual learning introduces modest FPS overhead—most pronounced on the resource-constrained Jetson Nano—while containerized serving further reduces throughput. Together, these results quantify the trade-off between accuracy gains from meta-learning (and PPL) and the computational costs of adaptability and deployment for car rear signal classification in ADAS.

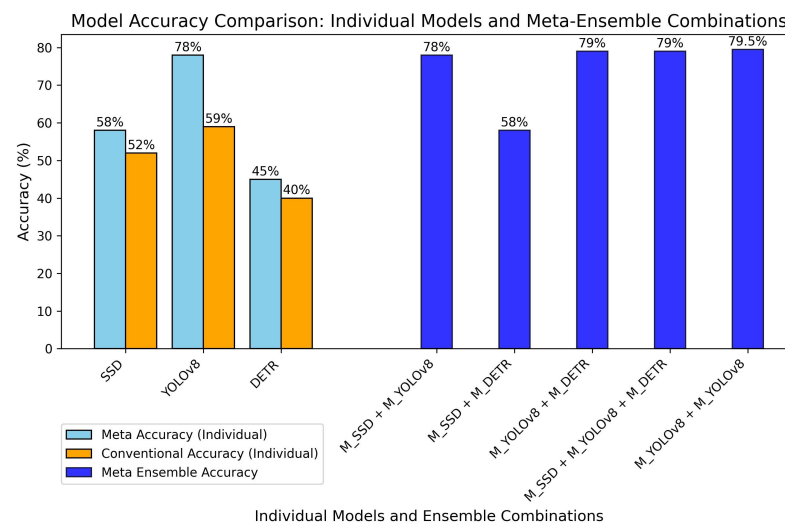


Figure 20. Accuracy comparison of individual models and meta model ensembles.

5.3.1. Meta Ensemble and Conventional Accuracies

Figure 20 provides a comparative analysis of the accuracy of individual models (SSD, YOLOv8, and DETR) evaluated under two configurations: meta model and conventional model. Additionally, the graph assesses the accuracy of ensembles formed exclusively from meta models, as conventional models demonstrated inferior performance, leading to the decision to focus on meta models for ensemble formation. The primary objective of this study is to investigate the performance advantages of meta model ensembles over individual models and evaluate their potential for improving accuracy in the specific task of car rear signal classification.

The accuracy of individual models is represented by two distinct bars for each model: one for meta accuracy and the other for conventional accuracy, depicted in light blue and orange, respectively. This clear distinction highlights the performance disparity between the two configurations. Across all individual models, meta models consistently outperform their conventional counterparts. Among the three models, YOLOv8 achieves the highest meta accuracy 78% (direct classification without PPL), demonstrating its robustness as a standalone model under the meta configuration.

The meta model ensembles, represented by dark blue bars, leverage the complementary strengths of individual meta models to achieve higher accuracy. These ensembles are

based on the principle that the ensemble accuracy is determined by the strongest contributing model. For instance, the ensemble comprising Meta-SSD and Meta-YOLOv8 achieves an accuracy of 78%, equal to the performance of Meta-YOLOv8, the more robust model in this pairing. Similarly, the ensemble involving Meta-SSD, Meta-YOLOv8, and Meta-DETR achieves a marginally higher accuracy of 79%, illustrating that combining multiple meta models offers slight improvements. However, the observed gain is incremental and does not represent a significant improvement over the strongest individual meta model.

Results show that YOLOv8 and its ensemble combinations perform better than SSD, DETR, and other ensemble methods, both with and without meta-learning. Its anchor-free detection head with multi-scale feature fusion enables precise localization of small, high-contrast objects such as taillights, while sustaining low-latency, real-time performance. By contrast, SSD relies on a fixed prediction grid and anchor priors, leading to degraded recall on small targets under variable scales and adverse weather conditions, as confirmed in our experiments. DETR's transformer-based decoding, though powerful, requires larger datasets, longer training cycles, and incurs additional overhead from Hungarian matching, all of which hinder its suitability for real-time inference.

Our results further show that meta model ensembles, while outperforming individual conventional models, remain less effective than a single Meta-YOLOv8 equipped with the PPL. Although ensembles can marginally exceed the performance of some individual meta models, their computational overhead outweighs the benefits, making them inefficient for resource-constrained ADAS deployments. For rear signal classification—particularly the disambiguation of LIS and RIS—individual high-performing meta models such as Meta-YOLOv8+PPL provide the best balance of accuracy, generalization, and efficiency. Taken together, these findings demonstrate that Meta-YOLOv8+PPL not only converges faster and generalizes better in few-shot settings but also aligns most closely with AITHENA's operational requirements for robust, real-time inference.

5.3.2. FPS

The comparative analysis of frames-per-second (FPS) performance across multiple devices provides critical insights into the compatibility of Advanced Driver Assistance Systems (ADAS) devices, the capability of cloud-based deployment, and the impact of continual learning mechanisms. The evaluation was conducted on diverse hardware platforms representing both edge and cloud environments: a PC with an Intel i5 processor (4 GB RAM), a Jetson Nano with a Maxwell GPU (commonly used in ADAS edge deployments), an MSI GP65 laptop with an i7 processor (32 GB RAM and RTX 2070 GPU), and the T4 GPU on Google Colab (a cloud-based system). Two scenarios were analyzed: FPS performance before and after continual learning (adaptability training) and before and after deployment using Flask with Docker containers. In the first scenario, continual learning methods introduced during adaptability training caused a slight reduction in FPS across all devices. For instance, the FPS on the PC dropped from 4 to 3, while the Jetson Nano, which is representative of edge-based ADAS devices, saw a reduction from 14 to 11. High-performance devices such as the MSI GP65 and T4 GPU exhibited similar trends, with FPS reductions from 30 to 27 and 51 to 47, respectively, see Figure 21a. These reductions result from the computational overhead associated with continual learning, where the model undergoes task-specific fine-tuning to improve its adaptability to real-world driving scenarios, such as car rear signal classification. While this process significantly enhances the model's accuracy and relevance to dynamic conditions, it also increases model complexity and inference time per frame, leading to a slight reduction in FPS. The Jetson Nano, as a resource-constrained platform, showed a more pronounced slowdown when the PPL was integrated, underscoring the challenge of deploying continual learning and

post-processing in lightweight ADAS hardware. In our experiments, the PPL added an additional latency of roughly 4–8 ms per frame, leading to a slight reduction in throughput across all tested platforms, including the Jetson Nano. This overhead remains modest and highlights the inherent trade-off between achieving higher classification accuracy and maintaining real-time performance on embedded ADAS hardware.

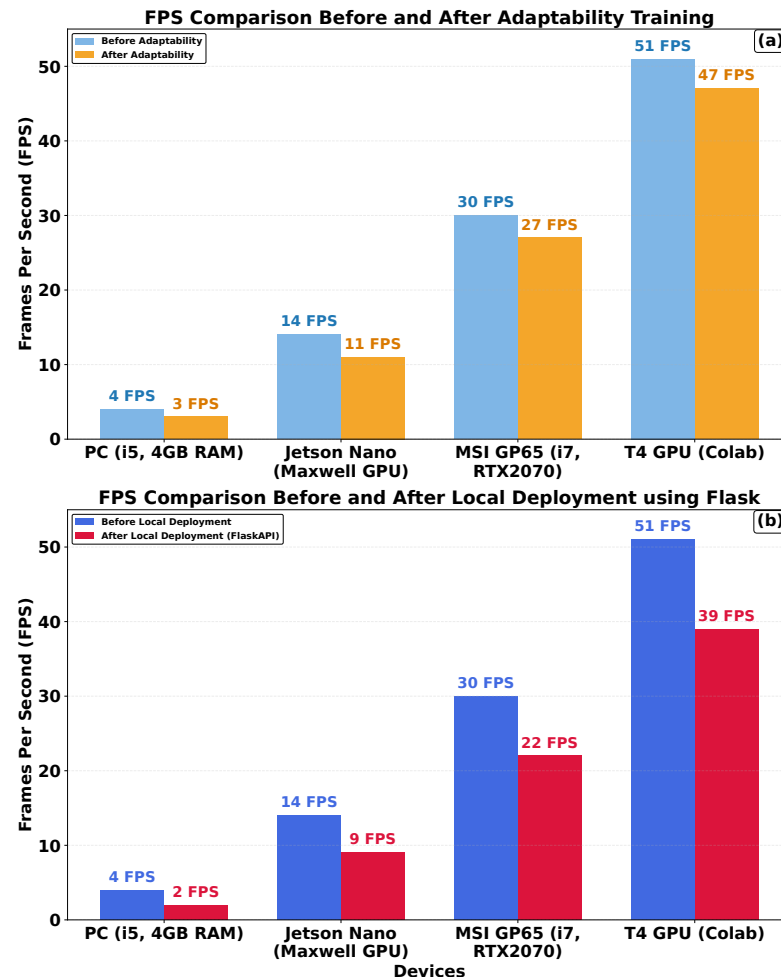


Figure 21. Comparison of inference rates for Meta-YOLOv8 with PPL across different devices. (a) FPS comparison before and after adaptability training on multiple hardware platforms. (b) FPS comparison before and after local deployment using a Flask API.

In the second scenario, cloud-based deployment using Flask with Docker containers further exacerbated the FPS drop across all devices. Compared to adaptability training, the performance reductions were more pronounced, with FPS on the PC dropping from 4 to 2 and the Jetson Nano experiencing a decrease from 14 to 9. Similarly, FPS on the MSI GP65 and T4 GPU declined from 30 to 22 and 51 to 39, respectively. The performance degradation in this scenario can be attributed to the combined overhead of Flask and Docker. Flask introduces latency through its request-response architecture, where incoming inference requests are queued, processed, and returned sequentially. Docker containers, while enabling portability and scalability for cloud deployment, add an additional abstraction layer that consumes computational resources, further reducing FPS; see Figure 21b. These effects are particularly limiting for resource-constrained ADAS devices like the Jetson Nano, where computational resources are already at a premium. On the other hand, high-performance cloud systems like the T4 GPU provide better scalability for cloud-based

ADAS applications, though they still suffer some performance degradation due to the inherent latency of containerized environments.

These findings emphasize the importance of balancing adaptability, edge compatibility, and cloud deployment efficiency in ADAS application pipelines. Continual learning mechanisms improve model adaptability and accuracy in dynamic conditions, making them highly suitable for ADAS applications. However, the associated computational costs necessitate optimization strategies to maintain real-time performance, particularly on edge devices. Additionally, while cloud deployment offers scalability and computational flexibility, mitigating the latency introduced by containerized environments is crucial for ensuring effective integration with ADAS. This analysis highlights the need to design hybrid deployment strategies that leverage the strengths of both edge and cloud environments while addressing the computational trade-offs inherent in continual learning and containerized deployment.

5.4. Sensitivity to Meta-Parameters

We initialized with a high learning rate (0.1) and momentum to quickly learn high-level features, then used an AutoKeras sweep to select 0.0085 and 0.935 for pre-training on task-similar traffic-light data, before refining on rear-signal data. Meta-learning, unlike standard fine-tuning, is designed for rapid adaptation with minimal retraining [8]. We assessed sensitivity to the inner-loop learning rate α (Equations (1) and (2)) and the outer-loop task count M , as described in Equation (1): $\alpha \in [0.1 - 0.0001]$ and momentum $\beta_1 \in [0.7, 0.99]$ with an optimum values 0.0085, 0.935 (close to the AutoKeras rate), and M varied as in Equation (1); stability was also checked across 3/5/8/10-shot episodes and learning rates (Figures 17 and 20). Across settings, peak F1 and mAP@0.5:0.95 changed only marginally and method ranking was unchanged—Meta-YOLOv8+PPL remained strongest. Larger M modestly smoothed F1–confidence curves (diminishing returns), while extreme α slowed adaptation or skewed precision–recall. For deployment, we retain the defaults in Section 3.2, choose the threshold at the F1–confidence peak, and report mAP for threshold-agnostic performance.

5.5. Additional Experiments with Recent YOLO Versions

Our experiments were initially conducted on YOLOv8 (late 2023–early 2024). Completion was delayed by extensive data versioning and ensembling studies. During this interval, newer YOLO releases (v9–v12) and real-time transformer-based detectors were introduced with substantial architectural changes. We carried out targeted comparisons against YOLOv8, assessing both accuracy (mAP) and throughput (FPS).

Empirically, with PPL enabled, YOLOv11 achieved a modest $\sim 1\%$ mAP gain, while several later variants realized only $\sim 0\text{--}1\%$ (and in some cases underperformed YOLOv8; see Figures 12 and 22). By contrast, using meta-learning without PPL, YOLOv9 and YOLOv12 improved by approximately $\sim 3\%$ and $\sim 2\%$, respectively; moreover, combining meta-learning with PPL yielded smaller gains than meta-learning alone. However, in real-time inference, YOLOv8 consistently achieved better FPS. Closer analysis indicated that these accuracy gains were driven mainly by BIS and Car Leaving classes, while car rear signal classification decreased by $\sim 2\text{--}3\%$ relative to YOLOv8 (both with and without PPL). Our interpretation is that, from v9 onward, hybrid CNN–Transformer backbones and their encoder–decoder pipelines can improve large-object recognition but may penalize small-object performance (e.g., left/right rear indicators) and reduce throughput, which is critical for our work.

Importantly, our research is not about proving that a particular model is “best.” Our focus is on a model-agnostic meta-learning (MAML) training methodology that can be

applied to any detector now or in the future. We emphasize training under data-scarce conditions and improving detection capability through meta-learning rather than relying on a specific architecture. This keeps the approach portable as models evolve. To keep the main paper concise, we provide the source code and full results for these additional experiments—with and without PPL—in our repository (<https://github.com/VasuTammi/setti/Real-Time-Camera-Based-Rear-Car-Signal-Detection-in-ADAS-Using-Meta-Learning/tree/main/Additional-Experiments>, accessed on 27 October 2025).

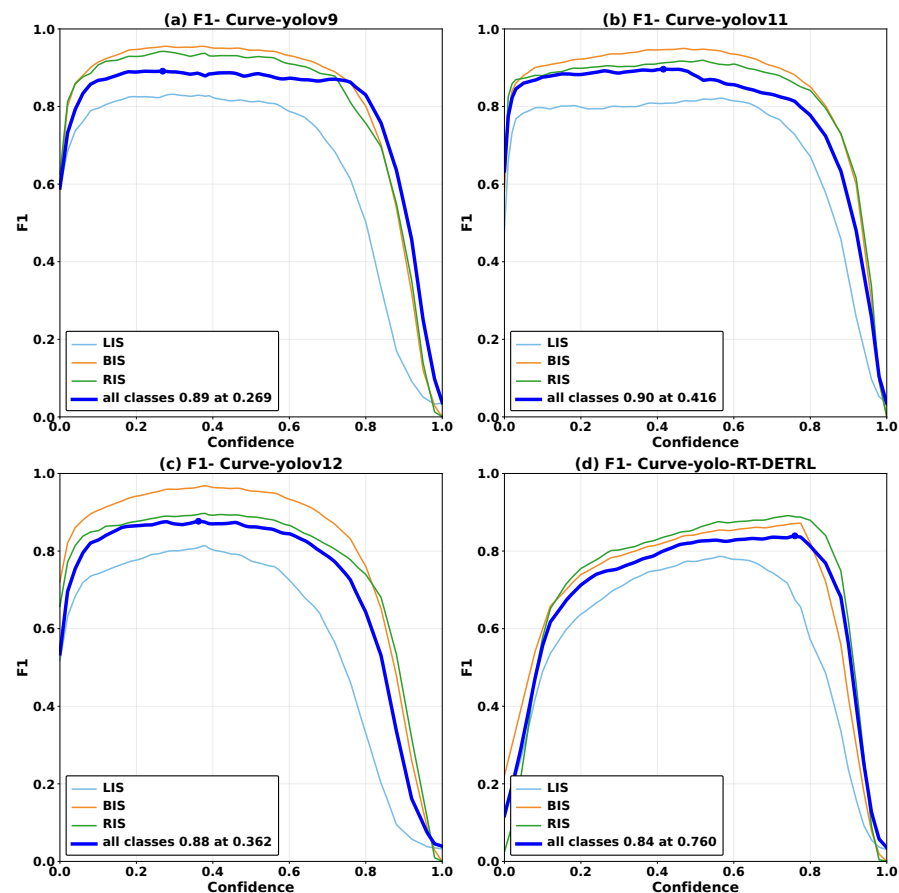


Figure 22. F1–Confidence curves with meta-learning (PPL) across recent YOLO versions.

6. Conclusions

This paper presented the Meta-YOLOv8+PPL framework, a novel approach for detecting rear car signals, designed to meet the challenges of ADAS in dynamic and resource-constrained environments. The proposed system addresses critical challenges in distinguishing left and right turn signals and brake lights under real-world complexities. The meta-learning paradigm enables robust generalization from limited datasets, while the PPL enhances classification accuracy through geometric and spatial heuristics, resolving ambiguities in visually similar signals.

Experimental results demonstrate the framework's superiority over conventional object detection models, achieving a precision of 89%, F1-score of 88%, and mAP@50 of 89%, with stable results across multiple runs. The model also maintains robust performance under adverse conditions, including low-light scenarios, rain, and fog, with reliable detection distances of 25–30 m and in normal conditions up to 120 m in a 75° detection arc. Comparative analyses confirm the Meta-YOLOv8+PPL's dominance over standalone and ensemble configurations, with diminishing returns observed for ensembles relative to computational overhead, solidifying the standalone model as optimal for real-time deployment.

Edge compatibility was validated through deployment on a NVIDIA Jetson Nano, confirming its scalability for in-vehicle applications. Few-shot learning experiments further underscore its adaptability, achieving rapid convergence with 40-image datasets (10 training and 3 validation samples per scenario). This capability, combined with mitigation of catastrophic forgetting and data scarcity, positions the framework as a practical solution for edge-based ADAS.

Integrating radar, LiDAR, or both as supporting modalities further strengthens the robustness, reliability, and safety of ADAS/AV rear-signal detection. Although these sensors do not directly interpret the chromatic or directional semantics of taillights, they provide fail-safe cues in safety-critical scenarios. Vision-based models such as Meta-YOLOv8+PPL excel at rear-signal classification under nominal conditions, but performance can degrade in heavy rain/snow, very low illumination, or severe occlusion. Radar/LiDAR mitigate these failure modes by adding redundancy, motion awareness (e.g., velocity tracking, early-braking cues), and high-resolution spatial mapping for precise localization with fewer false positives.

6.1. Future Scope

The integration of radar or LiDAR, along with Meta-YOLOv8 with PPL via heterogeneous sensor fusion, enables robust multi-modal validation of car rear signals (e.g., brake lights, turn indicators), ensuring uninterrupted functionality in highly complex environments. Future research will prioritize meta-learning-driven architectures to optimize fusion strategies, advancing geometric-semantic reasoning algorithms to accommodate regional signal design variations while optimizing computational-thermal efficiency for scalable edge-cloud deployments. This framework bridges critical accuracy-adaptability-efficiency trade-offs, advancing ADAS toward fail-safe autonomy and establishing next-generation vehicular safety protocols. However, detecting dynamic signaling patterns (e.g., sequential turn indicators, adaptive brake lighting) remains a challenge, with current models exhibiting limited robustness. Addressing these gaps requires adaptive spatio-temporal reasoning frameworks to interpret temporal signal evolution and lighting variations reliably. Enhancing these capabilities is pivotal for ensuring operational trustworthiness and holistic situational awareness in real-world driving scenarios, particularly in high-speed or occluded environments.

6.2. Limitations

Since our target detection objects are significantly more reflective compared to other detection tasks, the Meta-YOLOv8+PPL framework marks a significant step forward in car rear signal detection to increase situational awareness. However, certain limitations may hinder its effectiveness for deployment in real-world production environments. One major challenge is tied to constraints in edge deployment. Although the system has been successfully validated on a Jetson Nano and other low configuration devices, it demonstrates suboptimal real-time performance in high-speed scenarios, such as highway driving. This underscores the need for further optimization to reduce computational overhead or, alternatively, the integration of more powerful edge computing resources to handle the demands of such high-speed environments effectively. In addition to latency issues, the model struggles with intense glare, such as direct sunlight or strong headlight reflections. These conditions pose difficulties, particularly when detecting signals at distances greater than 25–30 m. Under such circumstances, the system's ability to differentiate colors and maintain detection accuracy diminishes, impacting its reliability in challenging lighting environments. Practical mitigations include INT8 TensorRT quantization and channel-

pruning for edge latency, upgrading to Jetson Orin for higher throughput, and hybrid fusion (camera+radar/LiDAR) to handle glare, heavy rain, and long-range cases.

Author Contributions: Conceptualization: V.T., M.P.C. and M.M.-S. Methodology: V.T. Software: V.T. Formal analysis: V.T. Resources: G.S. Writing (review and editing): V.T., G.S., M.P.C. and M.M.-S. Supervision: G.S., M.P.C. and M.M.-S. Project administration: V.T. and G.S. All authors have read and agreed to the published version of the manuscript.

Funding: This research was supported by Infineon Technologies AG (Munich, Germany) and the University of Granada (Spain). It was partially funded by the European Union’s Horizon Europe Research and Innovation Program through Grant Agreement No. 101076754 (Althena project), and by the Spanish Ministry of Economic Affairs and Digital Transformation (NextGenerationEU funds) through project IA4TES MIA.2021.M04.0008.

Data Availability Statement: Data is contained within the article and the accompanying code repository.

Conflicts of Interest: Vasu Tammiseti and Georg Stettinger were employed by Infineon Technologies AG. The remaining authors declare that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

References

1. Barbosa, F.M.; Osório, F.S. Camera-radar perception for autonomous vehicles and ADAS: Concepts, datasets and metrics. *arXiv* **2023**, arXiv:2303.04302. [\[CrossRef\]](#)
2. Sumalatha, I.; Chaturvedi, P.; Patil, S.; Thethi, H.P.; Hameed, A.A. Autonomous multi-sensor fusion techniques for environmental perception in self-driving vehicles. In Proceedings of the 2024 International Conference on Communication, Computer Sciences and Engineering (IC3SE), Gautam Buddha Nagar, India, 9–11 May 2024; pp. 1146–1151. [\[CrossRef\]](#)
3. Doshi, A.; Morris, B.; Trivedi, M. On-road prediction of driver’s intent with multimodal sensory cues. *IEEE Pervasive Comput.* **2011**, *10*, 22–34. [\[CrossRef\]](#)
4. Mallick, M.; Shim, Y.D.; Won, H.I.; Choi, S.K. Ensemble-Based Model-Agnostic Meta-Learning with Operational Grouping for Intelligent Sensory Systems. *Sensors* **2025**, *25*, 1745. [\[CrossRef\]](#) [\[PubMed\]](#)
5. Lee, J. D.; McGehee, D. V.; Brown, T. L.; Reyes, M. L. Collision Warning Timing, Driver Distraction, and Driver Response to Imminent Rear-End Collisions in a High-Fidelity Driving Simulator. *Hum. Factors* **2002**, *44*, 314–334. [\[CrossRef\]](#)
6. Li, D.; Yang, Y.; Song, Y.Z.; Hospedales, T. Learning to Generalize: Meta-Learning for Domain Generalization. *Proc. AAAI Conf. Artif. Intell.* **2018**, *32*, 3490–3497. [\[CrossRef\]](#)
7. Rakelly, K.; Shelhamer, E.; Darrell, T.; Efros, A.A.; Levine, S. Few-shot segmentation propagation with guided networks. *arXiv* **2018**, arXiv:1806.07373. [\[CrossRef\]](#)
8. Finn, C.; Abbeel, P.; Levine, S. Model-agnostic meta-learning for fast adaptation of deep networks. In Proceedings of the 34th International Conference on Machine Learning—PMLR, Sydney, Australia, 6–11 August 2017; pp. 1126–1135. [\[CrossRef\]](#)
9. Tammiseti, V.; Bierzynski, K.; Stettinger, G.; Morales-Santos, D.P.; Cuellar, M.P.; Molina-Solana, M. LaANIL: ANIL with Look-Ahead Meta-Optimization and Data Parallelism. *Electronics* **2024**, *13*, 1585. [\[CrossRef\]](#)
10. Tammiseti, V.; Stettinger, G.; Cuellar, M.P.; Molina-Solana, M. Meta-YOLOv8: Meta-Learning-Enhanced YOLOv8 for Precise Traffic Light Color Detection in ADAS. *Electronics* **2025**, *14*, 468. [\[CrossRef\]](#)
11. Chen, C.; Wang, G.; Peng, C.; Fang, Y.; Zhang, D.; Qin, H. Exploring rich and efficient spatial temporal interactions for real-time video salient object detection. *IEEE Trans. Image Process.* **2021**, *30*, 3995–4007. [\[CrossRef\]](#)
12. Li, L.; Wang, F.Y. Intelligent Vehicle Vision Systems. In *Advanced Motion Control and Sensing for Intelligent Vehicles*; Springer: New York, NY, USA, 2007; pp. 323–399. [\[CrossRef\]](#)
13. Tahir, N.U.A.; Zhang, Z.; Asim, M.; Chen, J.; ELAffendi, M. Object Detection in Autonomous Vehicles under Adverse Weather: A Review of Traditional and Deep Learning Approaches. *Algorithms* **2024**, *17*, 103. [\[CrossRef\]](#)
14. Wang, J.G.; Zhou, L.; Pan, Y.; Lee, S.; Song, Z.; Han, B.S.; Saputra, V.B. Appearance-Based Brake-Lights Recognition Using Deep Learning and Vehicle Detection. In Proceedings of the 2016 IEEE Intelligent Vehicles Symposium (IV), Gothenburg, Sweden, 19–22 June 2016; pp. 815–820. [\[CrossRef\]](#)
15. Islam, A.; Hossan, M.T.; Jang, Y.M. Convolutional neural networkscheme-based optical camera communication system for intelligent Internet of vehicles. *Int. J. Distrib. Sens. Netw.* **2018**, *14*, 1550147718770153. [\[CrossRef\]](#)
16. Arani, E.; Gowda, S.; Mukherjee, R.; Magdy, O.; Kathiresan, S.; Zonooz, B. A Comprehensive Study of Real-Time Object Detection Networks Across Multiple Domains: A Survey. *arXiv* **2022**, arXiv:2208.10895. [\[CrossRef\]](#)

17. Matin, M.A.; Fakhri, A.A.; Zaki, H.M.; Abidin, Z.Z.; Mustafah, Y.M.; Abd Rahman, H.; Mahamud, N.; Hanizam, S.; Rudin, N.A. Deep Learning-Based Single-Shot and Real-Time Vehicle Detection and Ego-Lane Estimation. *J. Soc. Automot. Eng. Malays.* **2020**, *4*, 61–72. [\[CrossRef\]](#)
18. Liu, B.; Zhao, W.; Sun, Q. Study of Object Detection Based on Faster R-CNN. In Proceedings of the 2017 Chinese Automation Congress (CAC), Jinan, China, 20–22 October 2017; pp. 6233–6236. [\[CrossRef\]](#)
19. Shuai, Q.; Wu, X. Object Detection System Based on SSD Algorithm. In Proceedings of the 2020 International Conference on Culture-Oriented Science & Technology (ICCST), Piscataway, NJ, USA, 28–31 October 2020; pp. 141–144. [\[CrossRef\]](#)
20. Liu, M.; Wang, X.; Zhou, A.; Fu, X.; Ma, Y.; Piao, C. UAV-YOLO: Small Object Detection on Unmanned Aerial Vehicle Perspective. *Sensors* **2020**, *20*, 2238. [\[CrossRef\]](#)
21. Khoei, A.G.; Yu, Y.; Feldt, R. Domain Generalization through Meta-Learning: A Survey. *Artif. Intell. Rev.* **2024**, *57*, 285. [\[CrossRef\]](#)
22. Guo, C.; Liu, H.; Chen, J.; Ma, H. Temporal Information Fusion Network for Driving Behavior Prediction. *IEEE Trans. Intell. Transp. Syst.* **2023**, *24*, 9415–9424. [\[CrossRef\]](#)
23. Arnold, S.M.R.; Mahajan, P.; Datta, D.; Bunner, I.; Zarkias, K.S. learn2learn: A Library for Meta-Learning Research. *arXiv* **2020**, arXiv:2008.12284. [\[CrossRef\]](#)
24. Shmelkov, K.; Schmid, C.; Alahari, K. Incremental Learning of Object Detectors without Catastrophic Forgetting. In Proceedings of the IEEE International Conference on Computer Vision (ICCV), Venice, Italy, 22–29 October 2017; pp. 3400–3409. [\[CrossRef\]](#)
25. Kao, C.W.; Wang, S.T.; Huang, C.S. Application of Edge Detection Technology Based on YOLOv8 in Smart Mobility Aids. In Proceedings of the 2024 IEEE 6th Eurasia Conference on Biomedical Engineering, Healthcare and Sustainability (ECBIOS), Yunlin, Taiwan, 15–17 November 2024; pp. 258–262.
26. Wang, G.; Chen, Y.; An, P.; Hong, H.; Hu, J.; Huang, T. UAV-YOLOv8: A Small-Object-Detection Model Based on Improved YOLOv8 for UAV Aerial Photography Scenarios. *Sensors* **2023**, *23*, 7190. [\[CrossRef\]](#) [\[PubMed\]](#)
27. Lin, T.Y.; Dollár, P.; Girshick, R.; He, K.; Hariharan, B.; Belongie, S. Feature Pyramid Networks for Object Detection. In Proceedings of the 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Honolulu, HI, USA, 21–26 July 2017; pp. 936–944. [\[CrossRef\]](#)
28. Terven, J.; Córdova-Esparza, D.M.; Romero-González, J.A. A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS. *Mach. Learn. Knowl. Extr.* **2023**, *5*, 1680–1716. [\[CrossRef\]](#)
29. Sohan, M.; Sai Ram, T.; Rami Reddy, C.V. A Review on YOLOv8 and Its Advancements. In *Cryptology and Network Security with Machine Learning*; Springer: Dordrecht, The Netherlands, 2024; pp. 529–545. [\[CrossRef\]](#)
30. Safaldin, M.; Zaghdien, N.; Mejdoub, M. An Improved YOLOv8 to Detect Moving Objects. *IEEE Access* **2024**, *12*, 59782–59806. [\[CrossRef\]](#)
31. Bochkovskiy, A.; Wang, C.Y.; Liao, H.Y.M. YOLOv4: Optimal Speed and Accuracy of Object Detection. *arXiv* **2020**, arXiv:2004.10934. [\[CrossRef\]](#)
32. Elfving, S.; Uchibe, E.; Doya, K. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural Netw.* **2018**, *107*, 3–11. [\[CrossRef\]](#)
33. Hussin, R.; Juhari, M.R.; Kang, N.W.; Ismail, R.; Kamarudin, A. Digital image processing techniques for object detection from complex background image. *Procedia Eng.* **2012**, *41*, 340–344. [\[CrossRef\]](#)
34. Prasad, D.K. Survey of the Problem of Object Detection in Real Images. *Int. J. Image Process. (IJIP)* **2012**, *6*, 441–458.
35. Sager, C.; Janiesch, C.; Zschech, P. A Survey of Image Labelling for Computer Vision Applications. *J. Bus. Anal.* **2021**, *4*, 91–110. [\[CrossRef\]](#)
36. Lee, Y.; Hwang, J.w.; Lee, S.; Bae, Y.; Park, J. An energy and GPU-computation efficient backbone network for real-time object detection. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops, Long Beach, CA, USA, 16–17 June 2019.
37. Chabi Adjobo, E.; Sanda Mahama, A.T.; Gouton, P.; Tossa, J. Automatic Localization of Five Relevant Dermoscopic Structures Based on YOLOv8 for Diagnosis Improvement. *J. Imaging* **2023**, *9*, 148. [\[CrossRef\]](#) [\[PubMed\]](#)
38. Ren, J.; Guo, Y.; Zhang, D.; Liu, Q.; Zhang, Y. Distributed and Efficient Object Detection in Edge Computing: Challenges and Solutions. *IEEE Netw.* **2018**, *32*, 137–143. [\[CrossRef\]](#)

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.